École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise

Rapport de Stage de Première Année

Analyse et modélisation prédictive des données du Port autonome de Dakar : une approche par Machine Learning

---

Stagiaire : **Mouhamed Nourou Dine Cisse**
Tuteur académique : **MERABET Massinissa**

Entreprise d'accueil : **Bangath Systems en partenariat avec Gainde 2000**
Adresse : **Liberté 6 Extension, Dakar Sénégal**
Encadrants professionnels : **KANE Arona Oumar**

Dates du stage : **26 Mai 2025 – 31 Juillet 2025**

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à **M. KANE Arona Oumar**, mon tuteur de stage chez Bangath Systems, pour l'opportunité qu'il m'a offerte, son soutien indéfectible et sa disponibilité permanente tout au long de cette expérience. Sa rigueur méthodologique, ses conseils avisés et ses réorientations m'ont été d'une aide précieuse pour mener à bien mes travaux.

Je souhaite également remercier tout particulièrement **M. MBODJ Mame Biram**, mon collègue au sein du pôle **Data Science**, pour sa collaboration active, ses échanges constructifs et sa bonne humeur quotidienne, qui ont grandement contribué à rendre ce stage enrichissant.

Enfin, je remercie chaleureusement l'ensemble de l'équipe de Bangath Systems pour leur accueil, leurs retours pertinents et l'ambiance de travail stimulante qu'ils ont su créer.

Table des matières

Table des matières 3

Abstract 1

Résumé 2

1 Introduction : Environnement de travail et stage 4
- 1.1 Bangath Systems . . . 4
- 1.2 Département de travail . . . 4
- 1.3 Contexte . . . 4
- 1.4 L'objectif du stage . . . 5
- 1.5 Contribution majeur . . . 5

2 Contexte et Préliminaires 6
- 2.1 Spécifications du stage . . . 6
- 2.2 Méthodes et outils utilisés dans l'entreprise . . . 6
- 2.3 Architecture logicielle cible . . . 7

3 Solution implémentée et ses contributions 8
- 3.1 Étude des solutions potentielles . . . 8
  - 3.1.1 Cas d'usage 1 : Classification de la probabilité de succès d'un dossier . . . 8
  - 3.1.2 Cas d'usage 2 : Prévision du volume de marchandises . . . 8
  - 3.1.3 Cas d'usage 3 : Prévision du nombre de dossiers . . . 9

4 Description des données 11
- 4.1 Présentation des données . . . 11
- 4.2 Sources des données . . . 11
- 4.3 Prétraitement des données . . . 11
- 4.4 Analyse exploratoire . . . 11
- 4.5 Visualisation . . . 12

5 Les cas d'usages 13
- 5.1 Méthode générale . . . 13
  - 5.1.1 Sélection des variables . . . 13
- 5.2 Estimation de la probabilité de succès d'un dossier . . . 13
- 5.3 Les séries temporelles . . . 17
- 5.4 Cas d'usage 2 : Prévision du volume de marchandises ou conteneurs . . . 19
- 5.5 Cas d'usage 3 : Prévision du nombre de dossiers . . . 28

5.5.1 Discussion sur les contributions 35

6 Conclusion et perspective **36**
6.1 Conclusion 36
6.2 Leçon apprise 36
6.3 Perspective 36

A DD&RS - Développement Durable et Responsabilité Sociétale **38**
A.1 Impacts environnementaux 38
A.1.1 Actions liées au stage et aux stratégies de l'entreprise 38

B Structure de l'entreprise **39**

C Détails techniques **40**
C.1 Extraits de code supplémentaires 40
C.1.1 Cas d'usage 1 : Estimation de la probabilité de succès d'un dossier 40

Bibliographie **45**

Abstract

This report presents the work carried out during an internship focused on data analysis, data enhancement, and the design of predictive models addressing three key business use cases. These include : classifying the success probability of a dossier using a Random Forest model ; forecasting the volume of goods using an ARIMA model ; and estimating the daily flow of created dossiers with the SARIMA algorithm. After thorough data preprocessing and exploratory analysis, the models were fine-tuned through cross-validation and evaluated using appropriate metrics (F1-score, RMSE, MAE). The obtained results are promising, with an F1-score of 0.99 for classification and a mean absolute error (MAE) of approximately 70% for the time series models.

Résumé

Ce rapport présente les travaux réalisés dans le cadre d'un stage axé sur l'analyse, la valorisation de données et la conception de modèles prédictifs répondant à trois problématiques métiers principales. Il s'agit de : la classification de la probabilité de succès d'un dossier à l'aide d'un modèle Random Forest, la prévision du volume de marchandises via un modèle ARIMA, et l'estimation du flux quotidien de dossiers créés à l'aide de l'algorithme SARIMA. Après un prétraitement rigoureux des données et une phase d'analyse exploratoire, les modèles ont été calibrés par validation croisée et évalués à l'aide de métriques adaptées (F1-score, RMSE, MAE). Les résultats obtenus sont prometteurs, avec notamment un F1-score de 0,99 pour la classification, et une erreur absolue moyenne (MAE) d'environ 70% pour les modèles de séries temporelles.

Glossaire

MAE Mean Absolute Error
ARIMA AutoRegressive Integrated Moving Average
SARIMA Seasonal ARIMA
RF Random Forest
APEX Oracle Application Express
PLSQL Procedural Language for SQL

Chapitre 1

Introduction : Environnement de travail et stage

1.1 Bangath Systems

Bangath Systems est une entreprise spécialisée en ingénierie des données et en intelligence artificielle. Elle offre des services de développement d'outils d'analyse, de visualisation et de valorisation de données. Son ambition est de proposer une solution complète d'acquisition, d'analyse et d'exploitation des données, en mettant à disposition des outils avancés basés sur le machine learning et l'intelligence artificielle.

Le stage s'inscrit dans le cadre d'un partenariat stratégique entre Bangath Systems, entreprise d'accueil, et Gaïndé 2000, fournisseur des données en collaboration avec le Port Autonome de Dakar, la Douane sénégalaise, le concessionnaire DP World et d'autres acteurs impliqués dans l'activité portuaire. L'objectif principal est d'analyser, valoriser et extraire des insights pertinents à partir des d onnées fournies, dans le but d'optimiser, d'accélérer et de fluidifier les activités portuaires.

1.2 Département de travail

J'ai été intégré au département **Data Science et Intelligence Artificielle**, qui a pour mission de :

- Collecter, nettoyer et analyser les données provenant de Gaindé 2000 ;
- Concevoir et tester des modèles prédictifs pour optimiser les processus logistiques ;
- Intégrer ces modèles dans des plateformes existantes, notamment via Oracle APEX et JupyterLab sur Oracle Cloud.

1.3 Contexte

GAINDE 2000 est un acteur majeur de la dématérialisation des procédures douanières particulièrement au Sénégal mais aussi dans la sous région, disposant d'un important patrimoine de données portuaires à travers sa plateforme Orbus Infinity, partenaire de ce projet. Dans un contexte où l'avènement de l'intelligence artificielle a rendu l'exploitation des données cruciale, leur valorisation devient un enjeu stratégique majeur.

1.4 L'objectif du stage

L'objectif principal de ce stage est d'analyser et de valoriser les données historiques fournies, concevoir, entraîner et évaluer des modèles d'intelligence artificielle pour la prédiction et l'analyse de flux logistiques. Plus précisément, il s'agit de :

- Analyser les données issues de la plateforme Orbus Infinity (marchandises, flux de dossiers, conteneurs, poids, etc.) ;
- Détecter les variables explicatives et construire des modèles prédictifs adaptés à chaque cas d'usage ;
- Implémenter ces modèles sous Python dans l'environnement Oracle Cloud ;
- Proposer un prototype ou une méthodologie reproductible intégrable à la plateforme de l'entreprise.

Afin de transformer ce patrimoine informationnel latent en outils d'aide à la décision, afin d'optimiser les processus métiers et de renforcer l'efficacité opérationnelle.

1.5 Contribution majeur

Les contributions principales du stage sont les suivantes :

- Un modèle de classification de la probabilité de succès d'un dossier, avec un **F1-score supérieur à 0.99** ;
- Des modèles de prévision du volume de (conteneurs, poids, colis, ...) (ARIMA, SARIMA, Random Forest), avec une **MAE inférieure à 0.36** ;
- Une analyse complète de séries temporelles (stationnarité, ACF/PACF, modélisation) ;
- Un pipeline d'analyse reproductible, documenté et adapté à l'infrastructure Oracle.

Chapitre 2

Contexte et Préliminaires

2.1 Spécifications du stage

Ce stage avait pour but de répondre à trois problématiques métiers identifiées par l'entreprise Bangath Systems en partenariat avec GAINDE 2000 :

- Estimer la probabilité qu'un dossier soit traité avec succès, à l'aide d'un modèle de classification (Random Forest) ;
- Prévoir le volume de marchandises ou conteneurs attendus à partir de données historiques portuaires ;
- Estimer le flux quotidien de dossiers afin d'optimiser la planification des ressources humaines.

Les exigences fonctionnelles comprenaient :

- L'amélioration de la précision des prédictions et la réduction des erreurs de modèles ;
- L'utilisation de métriques fiables (F1-score, MAE, RMSE) pour évaluer les performances ;
- L'intégration des modèles dans l'écosystème Oracle (notamment Oracle APEX et Oracle Cloud Data Science) ;
- L'analyse automatisée via un pipeline SQL/PLSQL compatible avec la base existante.

Au-delà des exigences fonctionnelles, le stage s'est déroulé dans un cadre collaboratif structuré. J'ai participé :

- aux **points quotidiens** d'avancement avec l'équipe de Bangath Systems ;
- à la **rédaction de rapports intermédiaires** sur l'évolution des résultats et les difficultés rencontrées ;
- à plusieurs **présentations techniques** auprès des partenaires du projet, afin de partager les résultats, justifier les choix de modélisation et recueillir des retours métier.

2.2 Méthodes et outils utilisés dans l'entreprise

L'environnement technique de l'entreprise repose largement sur la plateforme Oracle. La méthodologie suivie pendant le stage respectait les étapes classiques d'un pipeline de Data Science :

- **Extraction et nettoyage des données** : via requêtes SQL/PLSQL depuis Oracle APEX ;
- **Analyse exploratoire (EDA)** : réalisée dans des notebooks Jupyter via Oracle Cloud Data Science ;

— **Modélisation** : les modèles Random Forest ont été développés avec `scikit-learn`, et les séries temporelles avec `statsmodels` (ARIMA, SARIMA) ;
— **Automatisation** : un prototype d'API a été envisagé pour permettre des requêtes depuis Oracle APEX vers les notebooks Python hébergés dans le cloud.

Une phase de formation initiale m'a permis de me familiariser avec l'environnement Oracle, notamment avec l'interface d'Oracle Cloud Infrastructure, les notebooks Jupyter intégrés, ainsi que les spécificités de l'outil Oracle APEX.

2.3 Architecture logicielle cible

Le travail réalisé devait s'intégrer dans une architecture logicielle préexistante orientée Oracle. Plus précisément, les données étaient hébergées sur une base Oracle accessible via APEX, avec une partie traitement métier déjà partiellement automatisée via des scripts SQL.

Les modèles développés devaient donc être compatibles avec ce cadre technique. Un schéma de fonctionnement envisagé comprenait :

1. L'extraction des données via Oracle ;
2. L'envoi des données vers un environnement de calcul (Jupyter Notebook Oracle Cloud) ;
3. L'inférence du modèle pré-entraîné et le renvoi du résultat dans l'interface APEX ;
4. L'affichage ou l'utilisation directe de la prédiction dans l'outil métier.

Chapitre 3

Solution implémentée et ses contributions

3.1 Étude des solutions potentielles

Pour répondre aux trois problématiques métiers identifiées dans la section 1.3 (classification des dossiers douaniers, prévision du volume de marchandises, et prévision du nombre de dossiers), plusieurs approches ont été explorées pour chaque cas d'usage. Chaque solution a été évaluée en fonction de sa précision, sa rapidité, son interprétabilité, et sa faisabilité dans l'environnement technique de Bangath Systems (plateforme Oracle APEX et Oracle Cloud Data Science). Voici une analyse des solutions envisagées, avec leurs avantages et inconvénients.

3.1.1 Cas d'usage 1 : Classification de la probabilité de succès d'un dossier

Pour prédire si un dossier douanier sera régulier (traité sans blocage), plusieurs algorithmes d'apprentissage supervisé ont été testés :

- **Régression logistique** : Modèle linéaire simple, basé sur une fonction logistique pour estimer les probabilités.
- **SVM (Support Vector Machine)** : Algorithme cherchant à maximiser la marge entre les classes, adapté aux frontières complexes.
- **Random Forest** : Algorithme d'ensemble basé sur des arbres de décision, robuste au bruit et aux données déséquilibrées.
- **XGBoost** et **LightGBM** : Algorithmes de gradient boosting, optimisés pour les gros volumes de données.

3.1.2 Cas d'usage 2 : Prévision du volume de marchandises

Pour prévoir le nombre de conteneurs ou le volume de marchandises, des modèles adaptés aux séries temporelles et des approches basées sur l'apprentissage automatique ont été envisagés :

- **ARIMA** : Modèle statistique linéaire pour séries temporelles stationnaires, combinant composantes autorégressives et moyenne mobile.
- **SARIMA** : Extension d'ARIMA intégrant une composante saisonnière.
- **Random Forest** : Adaptation aux séries temporelles via l'ingénierie de variables temporelles (lags, moyennes mobiles, etc.).

3.1.3 Cas d'usage 3 : Prévision du nombre de dossiers

Pour estimer le flux quotidien de dossiers, les modèles suivants ont été explorés :
— **ARIMA** : Pour modéliser les tendances linéaires sans saisonnalité marquée.
— **SARIMA** : Pour capturer la saisonnalité hebdomadaire observée dans les données.
— **Random Forest** : Avec enrichissement des données par variables temporelles dérivées.

lightgray **Modèle**	**Avantages**	**Inconvénients**
Cas d'usage 1 : Classification		
Régression logistique	Simple à mettre en œuvre, rapide à entraîner, interprétable facilement	Sensible aux relations non linéaires, performances limitées sur données complexes
SVM	Efficace pour modéliser des frontières de décision complexes	Coût computationnel élevé, calibrage difficile, faible interprétabilité
Random Forest	Robuste face au bruit, capture les interactions non linéaires, relativement interprétable	Temps d'entraînement plus long, nécessite réglage des hyperparamètres
XGBoost / LightGBM	Très performant sur gros volumes, bonne capacité d'apprentissage	Complexité d'optimisation, moins interprétable, réglages délicats
Cas d'usage 2 : Prévision du volume de marchandises		
ARIMA	Méthode simple, adaptée aux séries stationnaires linéaires	Nécessite stationnarité, sensible aux ruptures et variations soudaines
SARIMA	Permet de modéliser la saisonnalité explicitement	Paramétrage complexe, limité par la taille du jeu de données
Random Forest	Flexible, capable de modéliser des relations non linéaires	Nécessite une ingénierie de variables conséquente, moins intuitif sur séries temporelles
Cas d'usage 3 : Prévision du nombre de dossiers		
ARIMA	Facile à implémenter et à comprendre	Ne modélise pas la saisonnalité hebdomadaire
SARIMA	Bien adapté à la saisonnalité, améliore la précision	Sensible au faible volume de données, paramétrage complexe
Random Forest	Robuste au bruit, modélise des relations complexes	Nécessite création de variables dérivées spécifiques
Prophet	Intègre saisonnalité, jours fériés et événements spéciaux	Moins performant sur des données très bruitées

TABLE 3.1 – Synthèse des avantages et inconvénients des solutions envisagées selon les cas d'usage

Ces solutions ont été évaluées en fonction de leur adéquation aux problématiques métiers décrites dans la section 1.3. La classification vise à fluidifier le traitement des dossiers en identifiant les risques de blocage, tandis que les prévisions des volumes de marchandises et de dossiers permettent d'optimiser l'allocation des ressources et la planification. Les modèles Random Forest et SARIMA ont été retenus

pour leur robustesse et leur capacité à répondre aux besoins opérationnels, malgré certaines limites liées au volume de données.

Chapitre 4

Description des données

4.1 Présentation des données

Le jeu de données initial est composé de **110 260 enregistrements** répartis sur **136 variables**.
— **80 variables qualitatives** (catégorielles),
— **39 variables quantitatives** (numériques),
— **17 variables de type `datetime`**.
Le taux global de valeurs manquantes est estimé à environ **12 %**.

4.2 Sources des données

Les données fournies par **Gaïndé 2000** sont hébergées sur la plateforme **Oracle Cloud Infrastructure**, qui permet de centraliser l'ensemble du travail et d'éviter tout enregistrement local des données. Elles sont prétraitées au moyen de scripts **SQL/PLSQL**, afin de les nettoyer et de les structurer convenablement.

Une fois ce prétraitement effectué, les données sont accessibles via une **connexion sécurisée** depuis un **notebook Jupyter** déployé dans l'environnement **Oracle Cloud Data Science**.

4.3 Prétraitement des données

Un ensemble d'opérations de nettoyage, de normalisation et de transformation des données a été appliqué pour assurer la qualité et la cohérence du jeu de données. Les étapes incluent notamment :
— Le traitement des valeurs manquantes (environ 12% des données),
— La standardisation des formats de date et d'heure,
— Le renommage et la sélection des variables pertinentes,
— L'ingénierie de nouvelles variables dérivées.

4.4 Analyse exploratoire

Des analyses statistiques ont été menées afin de mieux comprendre la structure du jeu de données, la distribution des variables, et les corrélations éventuelles. Cette étape a permis de guider la sélection des variables pour chaque cas d'usage, et de détecter des comportements ou schémas intéressants à modéliser.

Un échantillon de la matrice de corrélation

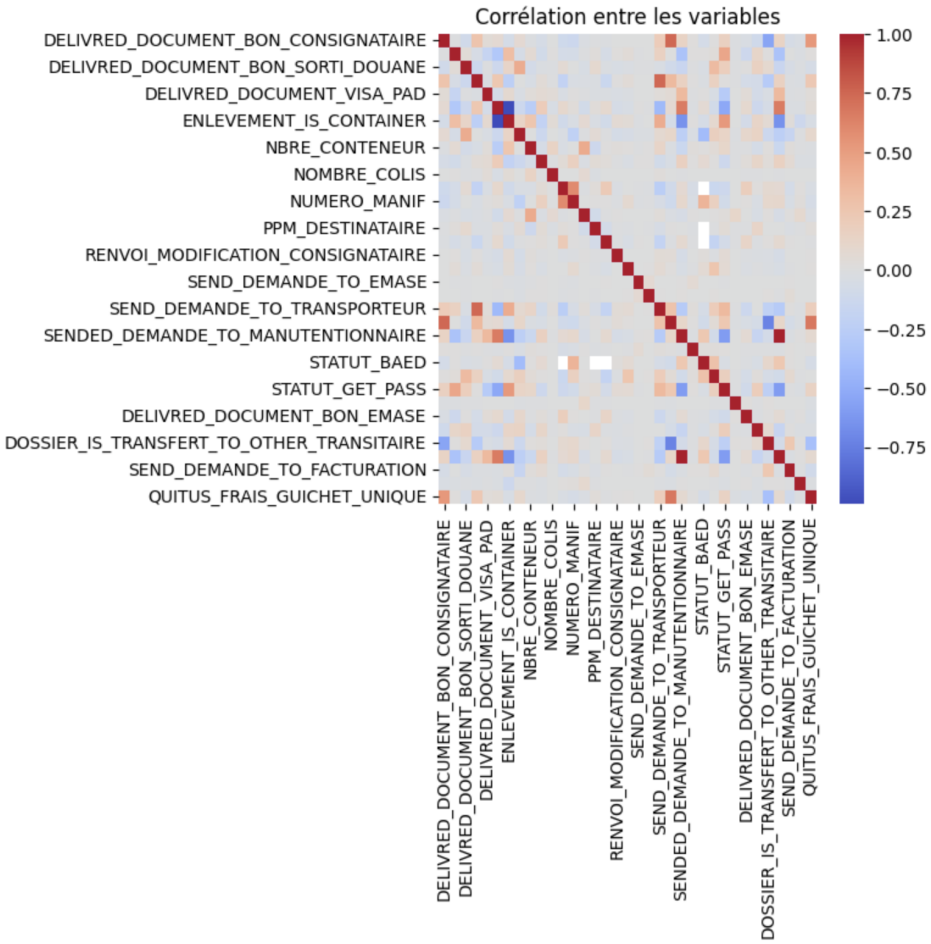


FIGURE 4.1 – Matrice de corrélation des variables

4.5 Visualisation

Des graphiques ont été utilisés pour compléter l'analyse exploratoire : histogrammes, courbes temporelles, heatmaps de corrélations, etc. Ces visualisations permettent de mieux appréhender les dynamiques internes des données et d'identifier d'éventuelles anomalies ou tendances.

Chapitre 5

Les cas d'usages

5.1 Méthode générale

5.1.1 Sélection des variables

Après une première analyse exploratoire des données, chaque cas d'usage a fait l'objet d'une identification spécifique de la variable cible. Nous avons ensuite tracé les matrices de corrélation pour mettre en évidence les variables explicatives les plus discriminantes. Une analyse approfondie de leur relation avec la cible a permis de restreindre le périmètre des variables pertinentes. Sur cette base, plusieurs modèles adaptés ont été envisagés, calibrés et comparés.

5.2 Estimation de la probabilité de succès d'un dossier

Description du problème

Nous cherchons à développer un système intelligent capable d'estimer la probabilité que chaque déclaration douanière soit traitée sans blocage ni retour pour correction.

En pratique, certains dossiers passent sans encombre, tandis que d'autres sont bloqués pour cause de pièces manquantes ou d'informations erronées, générant des retards et des coûts. L'objectif est donc de prédire, pour chaque dossier, la probabilité qu'il soit *régulier* (traité sans blocage) afin de :

— Mettre en avant ceux qui nécessitent une attention urgente ;
— Éviter que des dossiers à fort risque stagnent inutilement ;
— Allouer efficacement les agents et les ressources.

Cette vision préventive permet de fluidifier le processus, de réduire les délais et d'optimiser la chaîne de traitement.

Choix des modèles

Étant donné que nous disposons d'un historique labellisé (variable `DOSSIER_IS_REGULE` $\in \{0, 1\}$), le problème relève naturellement de l'apprentissage supervisé. Nous avons exploré plusieurs approches afin de trouver le meilleur compromis entre précision, rapidité et interprétabilité :

— **Régression logistique** : simple, rapide et interprétable ;
— **SVM** (Support Vector Machine) : efficace pour des frontières complexes, mais coûteux ;
— **Random Forest** : robuste au bruit, peu sensible au sur-apprentissage ;
— **XGBoost** et **LightGBM** : très performants sur de gros volumes de données.

Chaque modèle a été **calibré** (méthodes de Platt, régression isotone) afin que les probabilités produites reflètent fidèlement les fréquences observées.

Exploration des données (EDA)

Structure du jeu de données

- **Nombre total de dossiers** : 110 260
- **Variables** : 33
  - Variables numériques : 39 (ex. poids, nombre d'articles, délais...)
  - Variables booléennes : 80 (présence de documents, statuts...)
  - Variables temporelles : 17 (dates de soumission, validation...)
- **Valeurs manquantes** : environ 12 %
- **Variable cible** : `DOSSIER_IS_REGULE` $\in \{0, 1\}$

Nettoyage et transformation

- Suppression des colonnes constantes ;
- Uniformisation des formats de date et de nombre ;
- Transformation des champs de type document/statut en indicateurs binaires.

Exemple de variable discriminante Une des variables intéressantes est `PAYED_FRAI_BAD`. La distribution montre que les dossiers réguliers ont généralement réglé les frais BAD, ce qui confirme la pertinence de cette variable dans la prédiction.

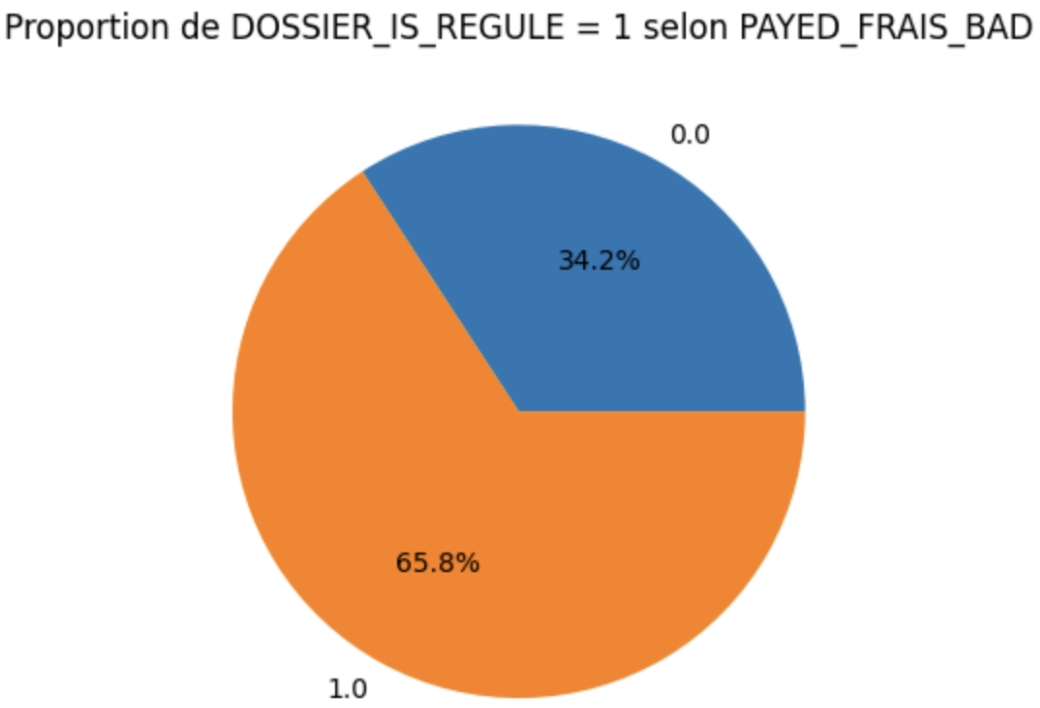


FIGURE 5.1 – Distribution de la variable PAYED_FRAI_BAD

Modélisation et évaluation

Stratégie

— Séparation des données : 80 % entraînement, 20 % test
— Validation croisée pour le réglage des hyperparamètres (nombre d'arbres, profondeur, etc.)
— Calibration des probabilités (méthodes de Platt et régression isotone)

L'algorithme Random Forest Le modèle Random Forest a donné les meilleurs résultats globaux pour notre tâche de classification. Il s'agit d'un algorithme d'ensemble basé sur la construction d'un grand nombre d'arbres de décision, puis sur l'agrégation de leurs prédictions. Ses avantages sont nombreux :

— **Robustesse** face au bruit et aux valeurs aberrantes ;
— **Rapidité** d'entraînement et de prédiction, grâce à l'entraînement parallèle des arbres ;
— **Capacité à capturer des interactions non-linéaires** entre les variables explicatives sans nécessiter de transformation préalable des données ;
— **Moins de surapprentissage** que les arbres individuels, grâce à la moyenne des prédictions ;
— **Interprétabilité** via l'importance des variables (feature importance) ;
— **Prédictions probabilistes fiables** après calibration.

Principe de la Random Forest Une Random Forest construit N arbres de décision $\{T_1, T_2, \ldots, T_N\}$ à partir d'échantillons bootstrapés du jeu de données d'entraînement. Chaque arbre est entraîné indépendamment à partir d'un échantillon aléatoire (avec remise) du jeu initial.

À chaque nœud de l'arbre, au lieu de considérer toutes les variables, on sélectionne aléatoirement un sous-ensemble de m variables (avec $m < d$, où d est le nombre total de variables explicatives). Cela permet de créer une diversité entre les arbres et de réduire la corrélation entre eux.

Prédiction Pour une observation x, chaque arbre T_i produit une prédiction $T_i(x)$. En classification binaire, la prédiction finale $\hat{p}(x)$ est obtenue par vote majoritaire ou moyenne des probabilités :

$$\hat{p}(x) = \frac{1}{N} \sum_{i=1}^{N} T_i(x)$$

La classe prédite est alors :

$$\hat{y}(x) = \begin{cases} 1 & \text{si } \hat{p}(x) \geq \tau \\ 0 & \text{sinon} \end{cases}$$

où τ est un seuil de classification (par défaut : $\tau = 0.5$).

Critère de division Chaque arbre est construit de manière récursive en sélectionnant, à chaque nœud, la variable X_j et le seuil s qui maximisent un critère de pureté. Le critère le plus courant est l'indice de Gini :

$$G(t) = 1 - \sum_{k=1}^{K} p_k^2$$

où p_k est la proportion d'exemples de classe k dans le nœud t. L'objectif est de minimiser la somme pondérée des impuretés des nœuds fils.

Évitement du surapprentissage Grâce à l'utilisation d'échantillons bootstrapés et de sous-ensembles de variables, la Random Forest évite naturellement l'overfitting, contrairement à un arbre unique très profond. La variance est réduite sans augmentation significative du biais.

Résultats

Calibration et niveaux de risque

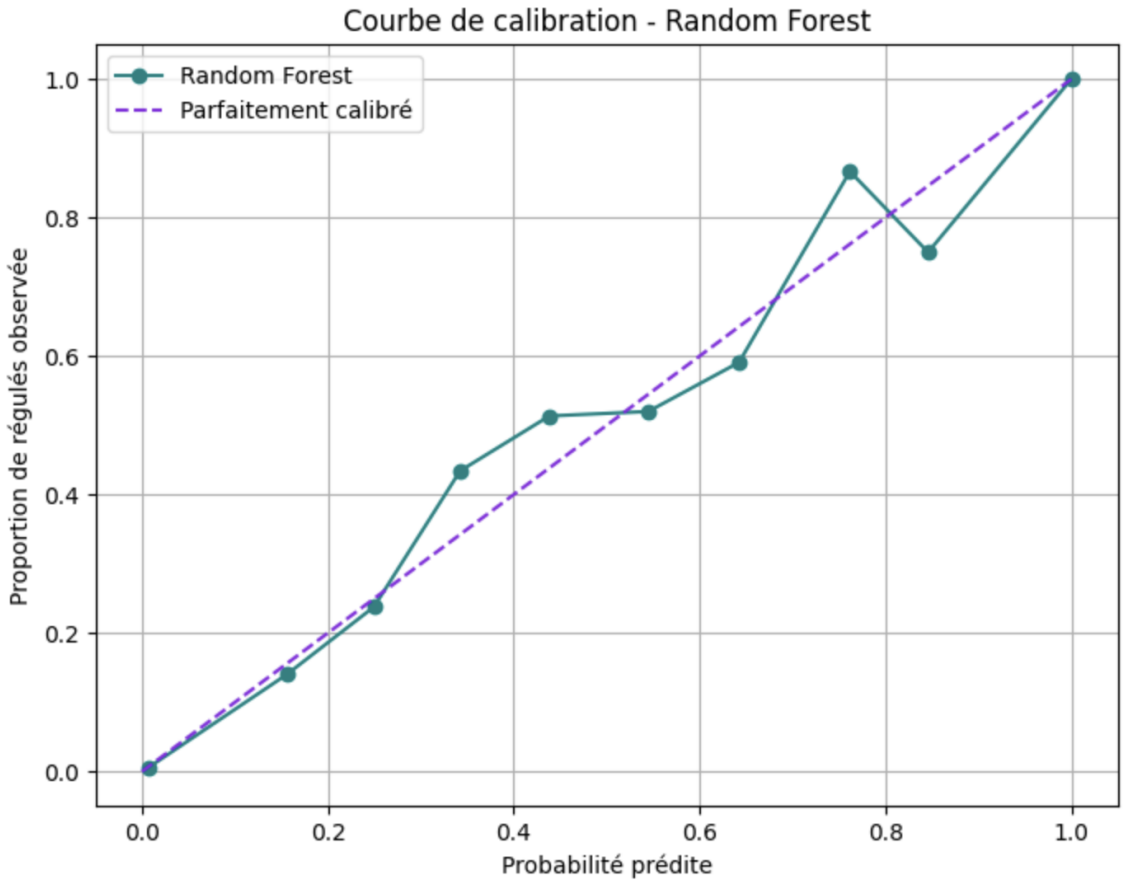


FIGURE 5.2 – Courbe de calibration des probabilités

— **Dossiers à faible risque** (proba $\geq 80\,\%$) : 99 % sont réellement régularisés.
— **Dossiers à haut risque** (proba $\leq 50\,\%$) : 97 % sont effectivement bloqués.
Ces résultats valident l'utilisation de seuils de 80 % et 50 % pour prioriser les vérifications manuelles.

	Précision	Rappel	F1-score
Régulier (1)	92.2 %	95.2 %	93.6 %
Bloqué (0)	99.8 %	99.6 %	99.7 %
Global		**96 %**	

TABLE 5.1 – Rapport de classification sur l'échantillon test

Rapport de classification

Conclusion Nous avons conçu un système prédictif robuste et calibré pour estimer, en amont, la probabilité qu'un dossier douanier soit régularisé sans blocage. Les apports majeurs sont :

— La détection précoce des dossiers à fort risque, permettant une action ciblée ;
— La fluidification des traitements pour les dossiers à faible risque ;
— L'optimisation des ressources humaines grâce à une hiérarchisation des priorités.

5.3 Les séries temporelles

Une **série temporelle** est une suite d'observations $\{y_t\}$ indexée par le temps t, où les données sont collectées à intervalles réguliers (journaliers, mensuels, annuels, etc.). Contrairement aux données classiques, l'ordre chronologique est fondamental dans les séries temporelles, car chaque observation peut dépendre des précédentes.

L'objectif de l'analyse des séries temporelles peut être :

— la **modélisation** de la dynamique du phénomène ;
— la **prédiction** des valeurs futures ;
— la **compréhension** des structures sous-jacentes (saisonnalité, tendance, bruit, etc.).

Exemple de série temporelle

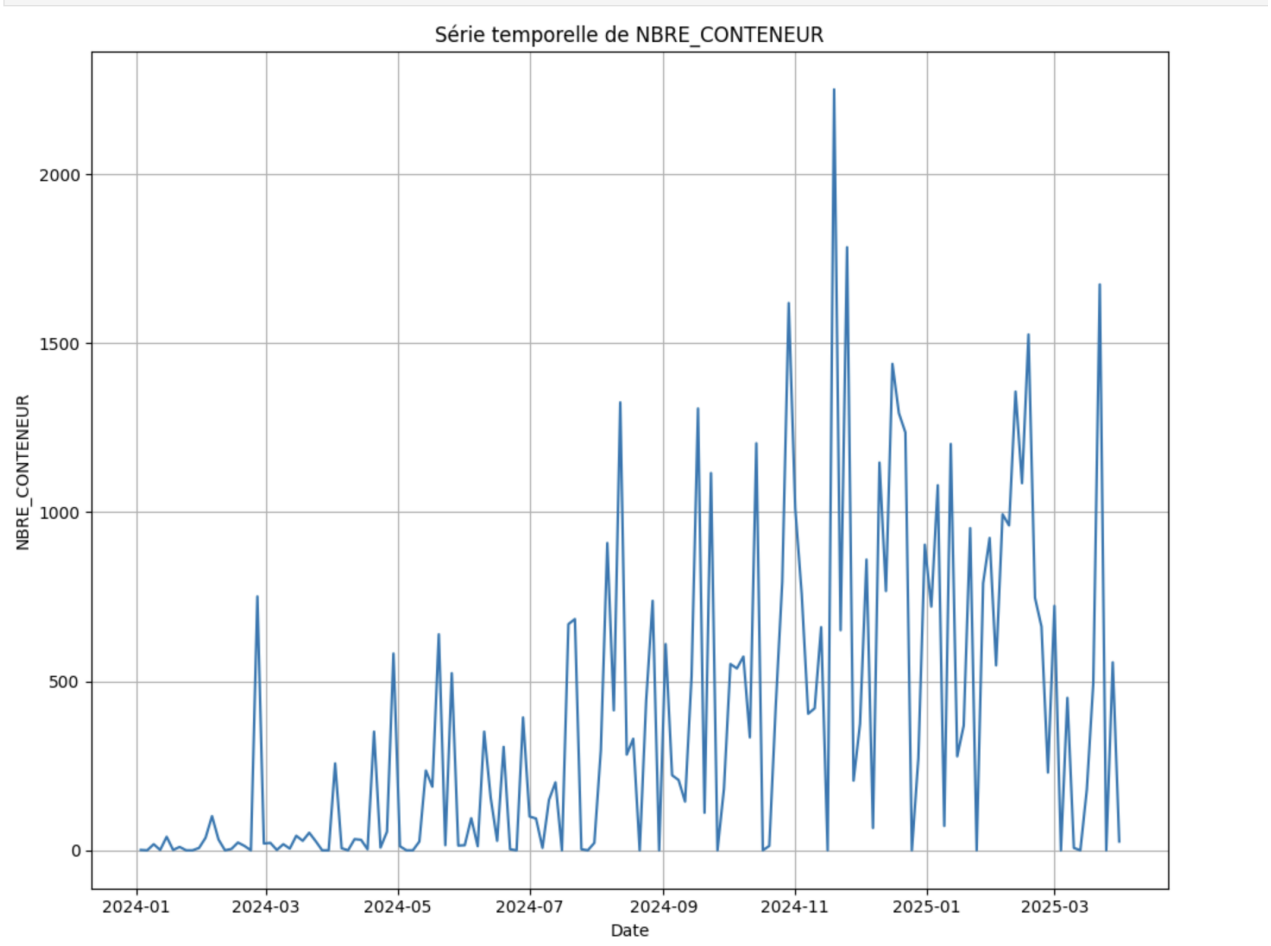


FIGURE 5.3 – Exemple de série temporelle

Étapes classiques de traitement

Méthode générale L'analyse d'une série temporelle suit généralement les étapes suivantes :

1. **Prétraitement des données** :
   — Remplissage des valeurs manquantes, suppression des valeurs aberrantes ;
   — Conversion du temps au bon format et vérification de la fréquence ;
   — Transformation éventuelle : log, différenciation, etc.
2. **Visualisation et décomposition** :
   — Décomposition en composantes : tendance (*trend*), saisonnalité (*seasonality*) et bruit (*residual*) ;
   — Outils : STL, décomposition classique additive ou multiplicative.
3. **Vérification de la stationnarité** :
   — Une série est dite stationnaire si sa moyenne, variance et autocorrélation sont constantes dans le temps ;
   — Tests statistiques : test de Dickey-Fuller augmenté (ADF), KPSS, etc. ;
   — En cas de non-stationnarité, on applique des transformations (différenciation, log).
4. **Analyse de la dépendance temporelle** :
   — Tracer l'**ACF** (fonction d'autocorrélation) pour observer la corrélation entre y_t et y_{t-k} ;
   — Tracer la **PACF** (fonction d'autocorrélation partielle) pour estimer la dépendance directe avec les retards.
5. **Choix du modèle** :
   — En fonction des propriétés de la série, on peut choisir un modèle :
      — **ARMA/ARIMA** : pour les séries linéaires et stationnaires ;
      — **SARIMA** : si saisonnalité présente ;
      — **VAR/VARMA** : pour les séries multivariées ;
      — **LSTM/RNN** : pour les structures non linéaires complexes.
   — Sélection des ordres (p, d, q) via les courbes ACF/PACF ou critère d'information (AIC/BIC).

Résumé visuel du pipeline

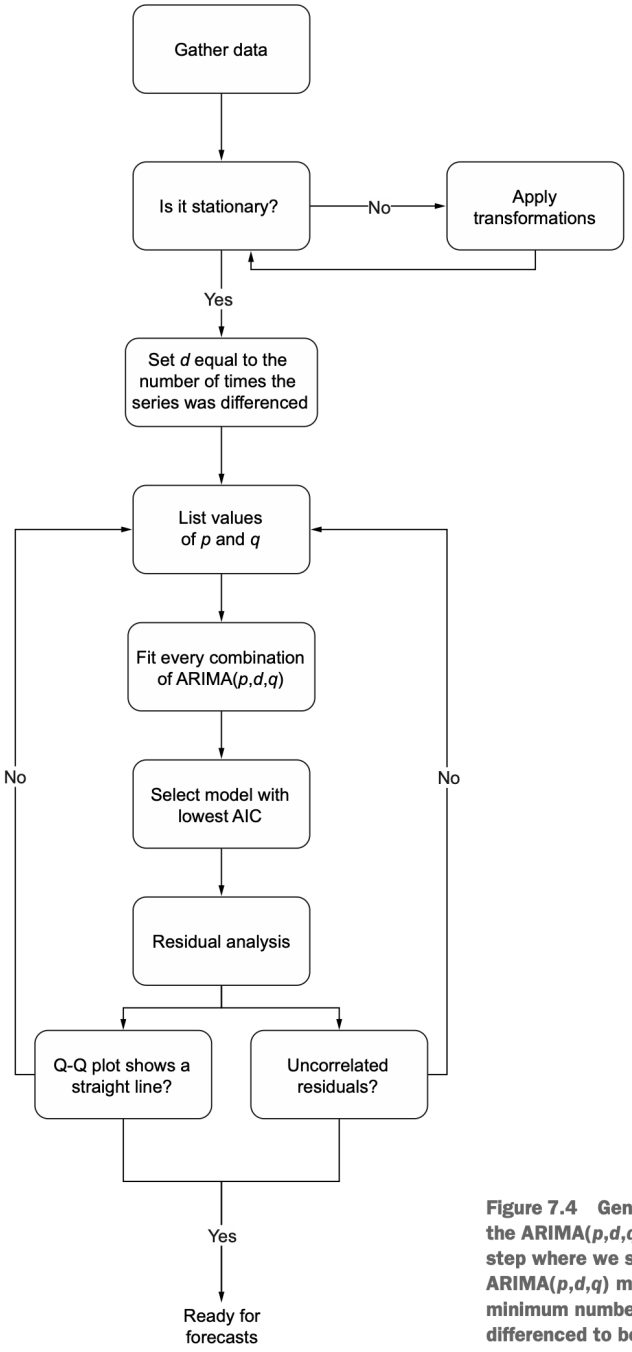


Figure 7.4 General modeling procedure for using the ARIMA(p,d,q) model. Notice the addition of a step where we specify the parameter d for the ARIMA(p,d,q) model. Here, d is simply the minimum number of times a series must be differenced to become stationary.

FIGURE 5.4 – Pipeline d'analyse des séries temporelles

5.4 Cas d'usage 2 : Prévision du volume de marchandises ou conteneurs

Objectif

Prédire le nombre de colis ou conteneurs attendus sur une période donnée afin d'optimiser l'allocation des quais, du personnel et des capacités de stockage, réduire les congestions et maîtriser les

coûts d'exploitation.

Gain attendu

- Réduction des files d'attente et des temps d'attente aux quais ;
- Meilleure planification des ressources humaines et matérielles ;
- Diminution des coûts liés aux sureffectifs ou aux pénuries de capacité.

Exploration des données (EDA)

Structure du jeu de données

- **Nombre d'observations** : 328
- **Fréquence** : par 3 jours
- **Nombre de variables** : 5
- **Variables cibles** :
  - `NBRE_CONTENEUR` (nombre de conteneurs)
  - `NOMBRE_COLIS` (nombre de colis)
  - `POIDS_TOTAL` (poids total en kg)
  - `NOMBRE_ARTICLE`

Création de variables temporelles Pour capturer les cycles saisonniers et hebdomadaires, nous avons enrichi chaque enregistrement avec :

- Jour de la semaine (`Lundi`, ..., `Dimanche`)
- Mois de l'année (`Janvier`, ..., `Décembre`)
- Le flux de marchandises est plus élevé en **janvier**, **juin** et **décembre**, suggérant des effets de saisonnalité (début, fin d'année, soldes).

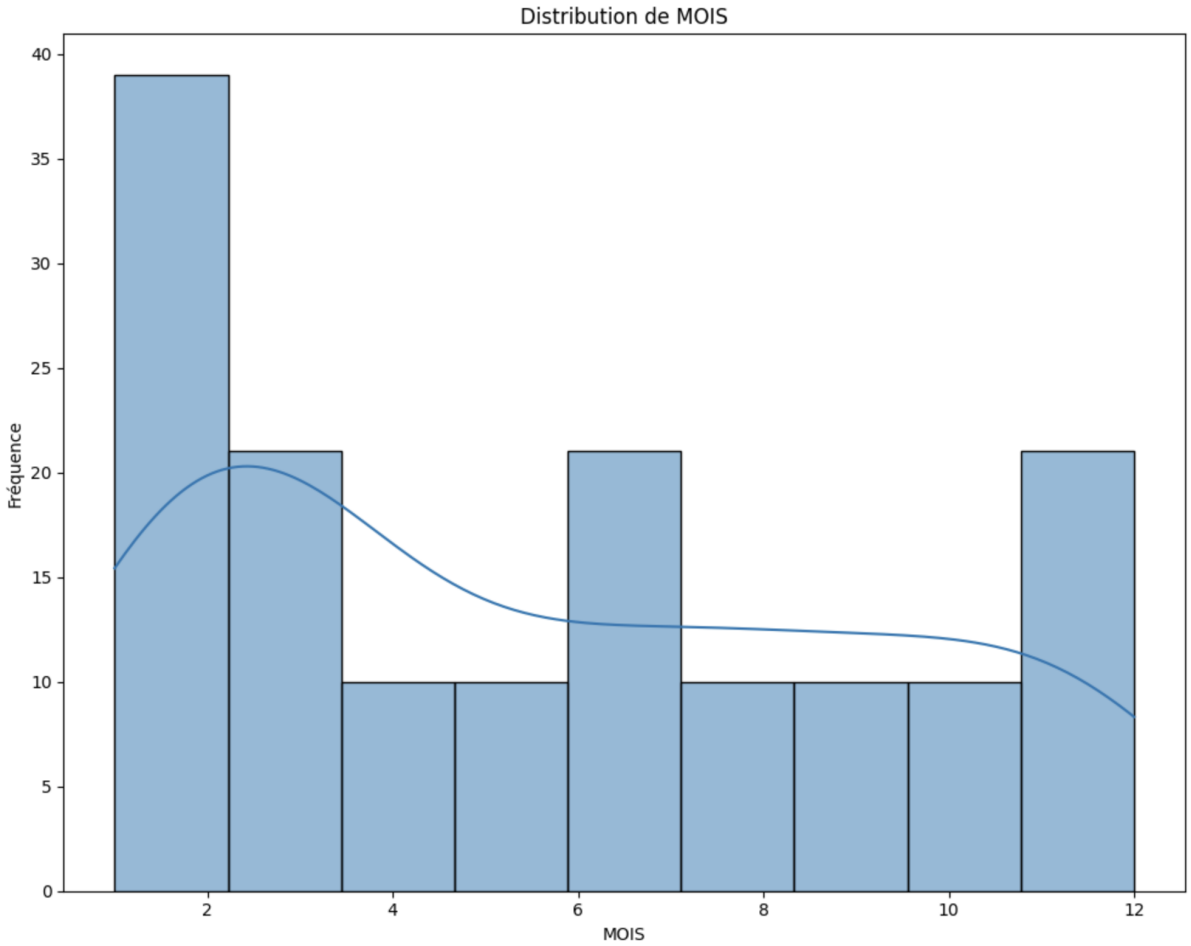


FIGURE 5.5 – Distribution mensuelle du nombre de conteneurs

Gestion des outliers Les données ne présentent pas d'outliers significatifs. Nous avons tout de même appliqué un traitement par la méthode IQR, mais les résultats sont restés inchangés. Nous continuerons donc sans suppression d'outliers.

Corrélation entre les variables cibles

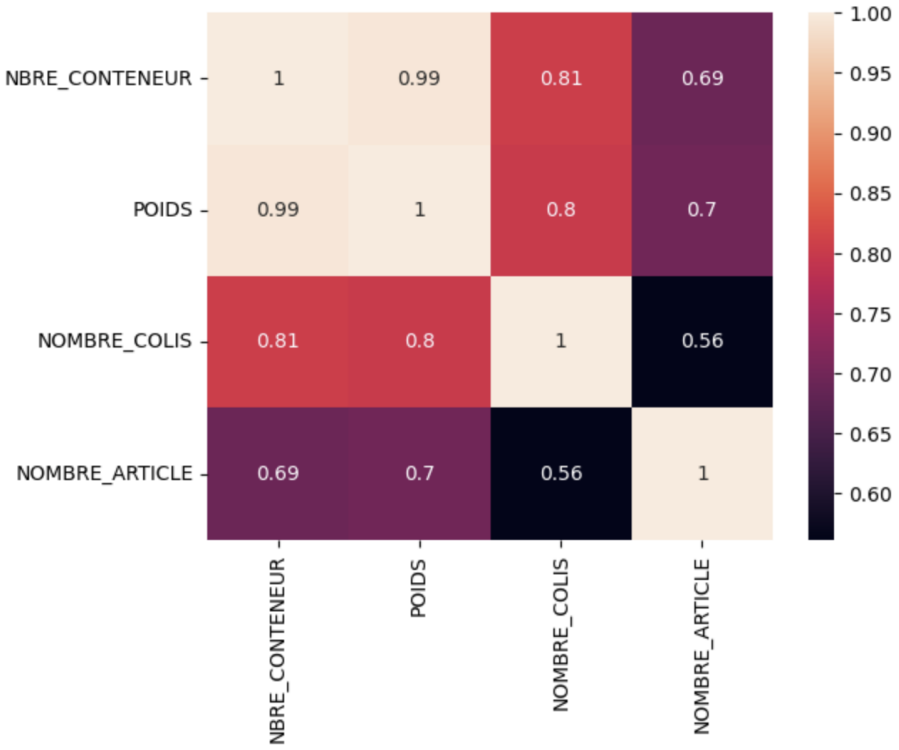


FIGURE 5.6 – Matrice de corrélation entre les variables cibles

On observe une corrélation positive marquée entre `NBRE_CONTENEUR`, `NOMBRE_COLIS` et `POIDS_TOTAL`, ce qui suggère qu'un modèle multivarié pourrait capturer efficacement ces interdépendances.

Visualisation et décomposition Les variables cibles présentent des tendances similaires lorsqu'elles sont tracées sur la même période. Par exemple, l'évolution du poids total sur des fenêtres de 3 jours met en évidence les mêmes pics saisonniers :

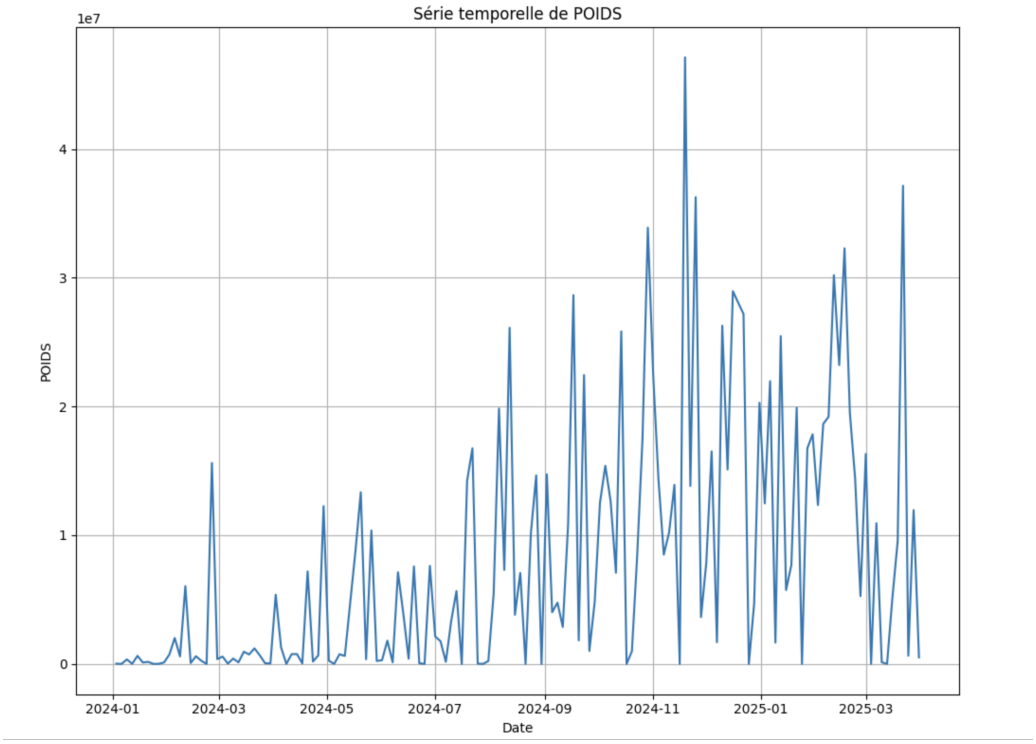


FIGURE 5.7 – Poids total des marchandises par fenêtre de 3 jours

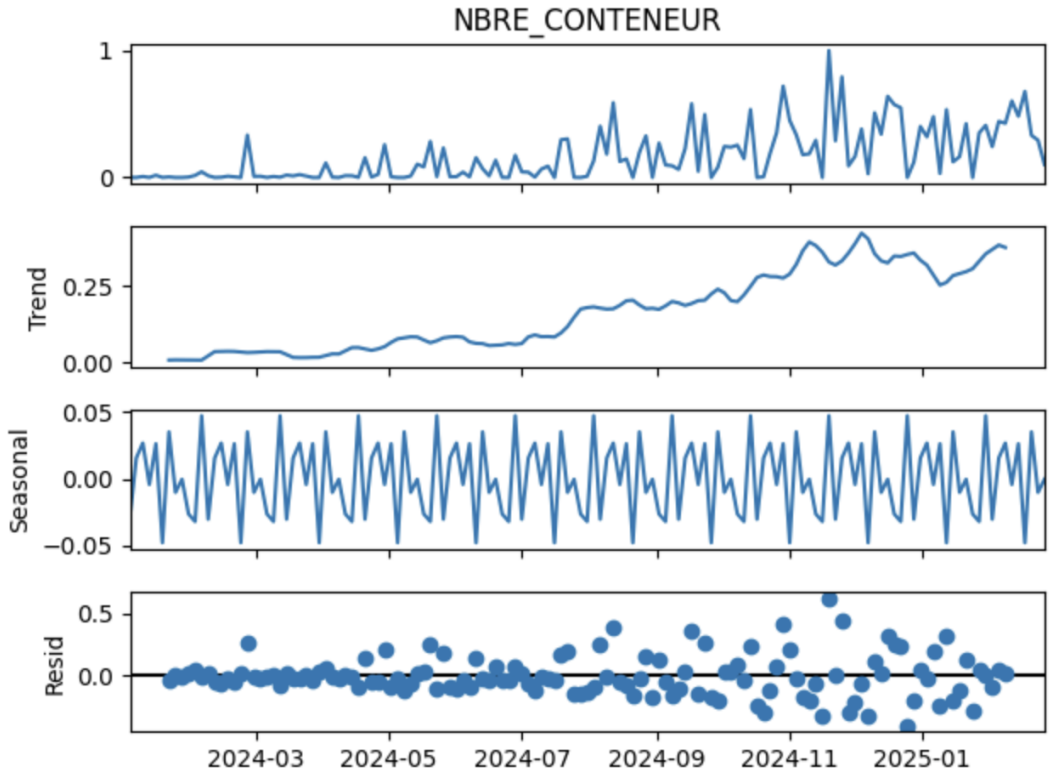
Pour mieux comprendre la structure temporelle, nous procédons à une décomposition additive en tendance, saisonnalité et résidu :

$$y_t = T_t + S_t + R_t.$$

Cette décomposition permet d'isoler :

- la **tendance** T_t, représentant l'évolution globale ;
- la **saisonnalité** S_t, capturant les cycles réguliers (hebdomadaire, mensuel) ;
- le **résidu** R_t, correspondant au bruit aléatoire et non modélisable.

Nous utilisons la méthode STL (Seasonal-Trend decomposition using Loess) pour obtenir ces composantes et guider le choix du modèle de prévision.




```
ADF statistic: -0.6376870220864491
p-value: 0.862236928625642
```

FIGURE 5.8 – Décomposition STL de la série temporelle

On observe une tendance croissante, la saisonnalité ne semble pas être régulière et les résidus sont distribués autour de zéro. On remarque d'ailleurs que la série n'est pas stationnaire du à un p-value $> 0{,}05$ du test ADF.

Tests de stationnarité Pour modéliser correctement une série temporelle, il est essentiel de vérifier sa **stationnarité** : une série est dite stationnaire si ses propriétés statistiques (moyenne, variance, autocorrélation) sont constantes dans le temps. Le test de Dickey–Fuller augmenté (ADF) permet de détecter la présence d'une racine unitaire.

Formulation du test ADF Le modèle de régression du test ADF s'écrit :

$$\Delta y_t = \alpha + \beta\, t + \gamma\, y_{t-1} + \sum_{i=1}^{p} \delta_i\, \Delta y_{t-i} + \varepsilon_t,$$

où :

- $\Delta y_t = y_t - y_{t-1}$ est la première différence de la série ;
- α est une constante (optionnelle) ;
- $\beta\, t$ est un terme de tendance (optionnel) ;
- γ est le paramètre clé :
  - Hypothèse nulle $H_0 : \gamma = 0$ (la série a une racine unitaire, $\Rightarrow$ non stationnaire) ;
  - Hypothèse alternative $H_1 : \gamma < 0$ (la série est stationnaire).

— Les termes $\sum_{i=1}^{p} \delta_i \, \Delta y_{t-i}$ permettent de corriger l'autocorrélation des résidus.

Si le p-value est inférieur à α, typiquement 0,05, on rejette H_0 et on conclut que la série est stationnaire.

Analyse de la dépendance temporelle

— On trace l'**ACF** (fonction d'autocorrélation) pour observer la corrélation entre y_t et y_{t-k} ;

— puis la **PACF** (fonction d'autocorrélation partielle) pour estimer la dépendance directe avec les retards.

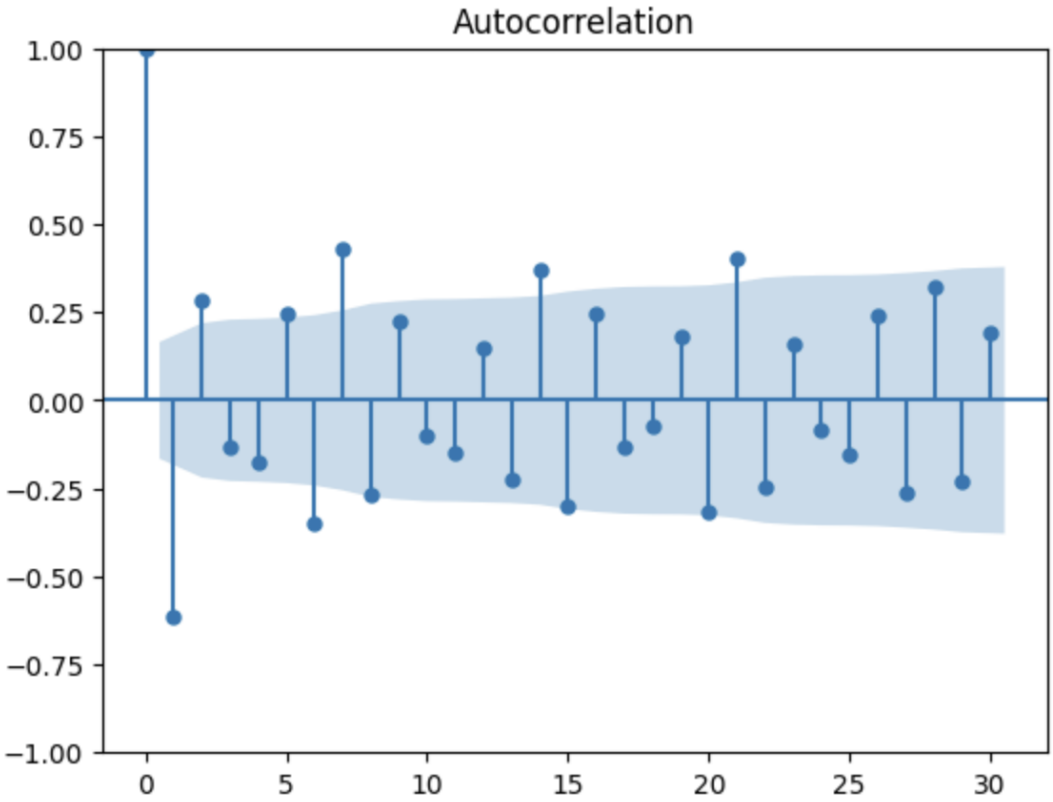


FIGURE 5.9 – Courbe d'autocorrélation des lags

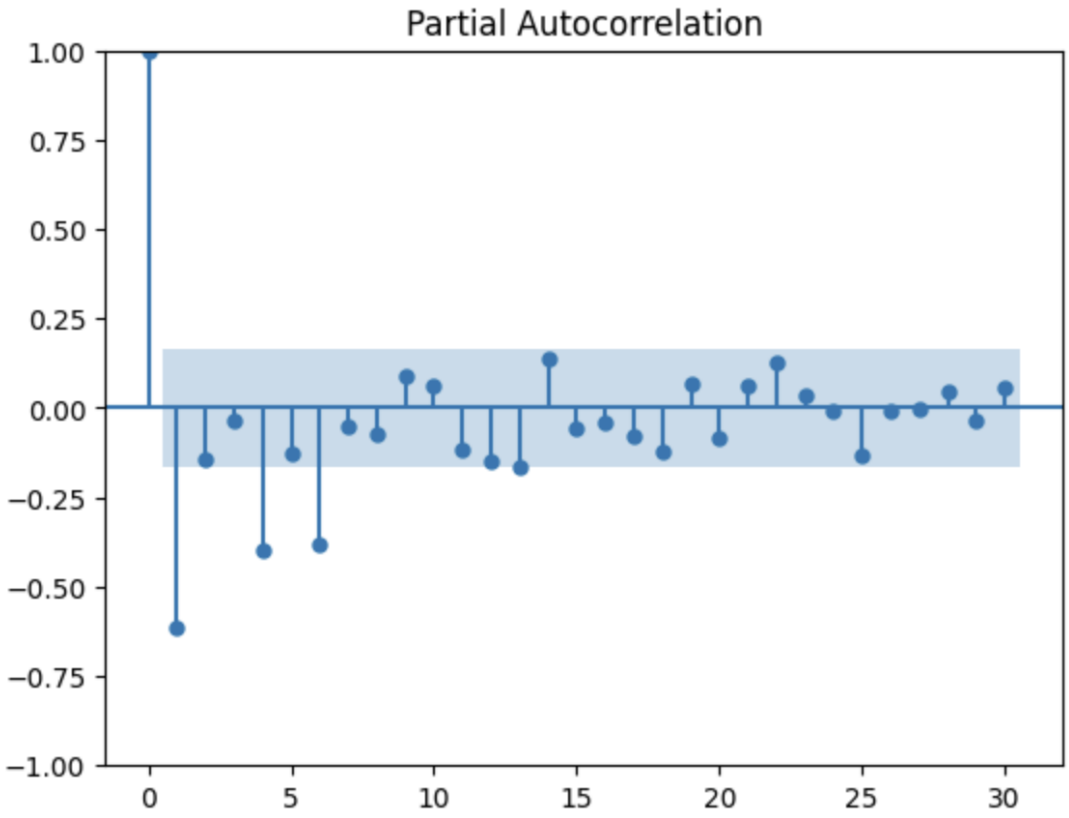


FIGURE 5.10 – Courbe d'autocorrélation partielle des lags

À partir de là, on peut passer à la modélisation.

Modélisation

Une fois que la série est rendue stationnaire, on cherche le modèle le plus adapté à cette dernière en interprétant les courbes ACF et PACF tracées ci-dessus.

Le modèle MA(q) : Moving Average Le processus MA(q) exprime y_t comme la moyenne μ plus la combinaison linéaire des q dernières erreurs et de l'erreur actuelle :

$$y_t = \mu + \varepsilon_t + \sum_{i=1}^{q} \theta_i \, \varepsilon_{t-i}, \quad \varepsilon_t \overset{\text{iid}}{\sim} (0, \sigma^2).$$

— Toujours stationnaire.
— L'autocorrélation $\rho(h)$ est non nulle uniquement pour $h \leq q$.
— Invertibilité : les racines de $\Theta(z) = 1 + \sum_{i=1}^{q} \theta_i z^i$ doivent être $|z| > 1$.

Le modèle AR(p) : AutoRegressive Le processus AR(p) relie y_t à ses p dernières valeurs :

$$y_t = \sum_{i=1}^{p} \phi_i \, y_{t-i} + \varepsilon_t, \quad \varepsilon_t \overset{\text{iid}}{\sim} (0, \sigma^2).$$

— Stationnarité si les racines de $\Phi(z) = 1 - \sum_{i=1}^{p} \phi_i z^i$ sont $|z| > 1$.
— La PACF s'annule pour $h > p$.

Le modèle ARIMA(p,d,q) Le modèle ARIMA combine différenciation et composantes AR et MA :

$$\Phi(L)\,(1-L)^d y_t = \Theta(L)\,\varepsilon_t,$$

avec $\Phi(L) = 1 - \sum_{i=1}^{p} \phi_i L^i$, $(1-L)^d$ la différence d'ordre d, $\Theta(L) = 1 + \sum_{j=1}^{q} \theta_j L^j$.

Sélection des ordres (p, d, q) On examine :
- ACF/PACF pour estimer p et q,
- Tests de stationnarité (ADF, KPSS) pour d,
- Critères d'information (AIC, BIC) : on choisit la combinaison (p, d, q) donnant le **plus petit AIC**.

Prévisions Pour la prévision du nombre de conteneurs, nous avons retenu le modèle ARIMA$(30, 1, 15)$ après sélection par minimisation de l'AIC. Sur l'échantillon de test, ce modèle a atteint une erreur absolue moyenne (MAE) de 0,36.

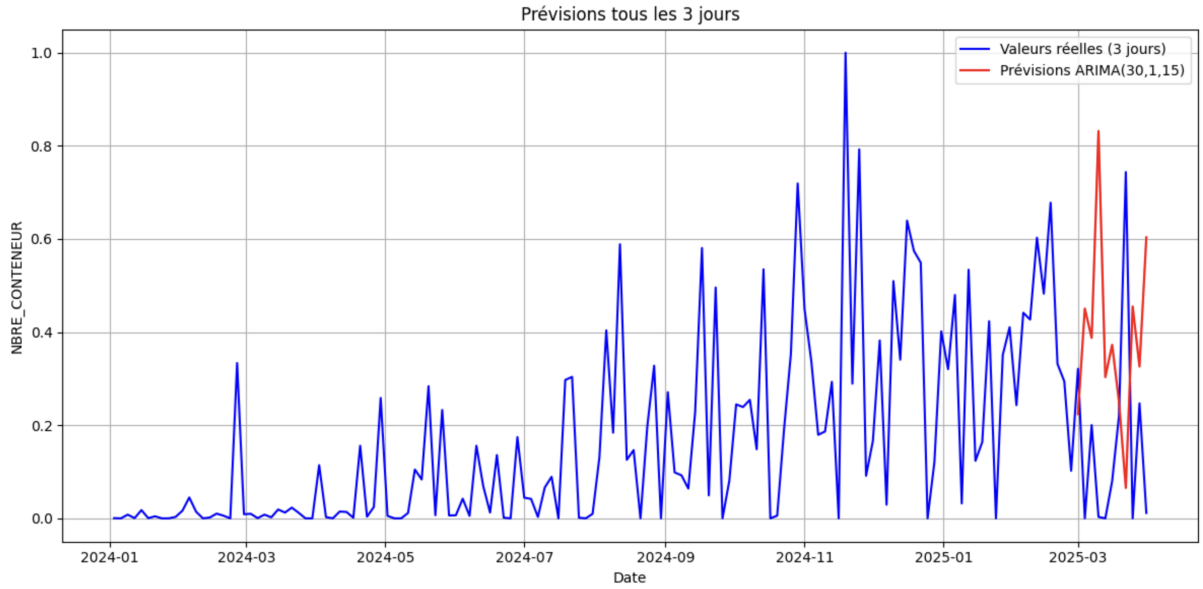


FIGURE 5.11 – Prévisions ARIMA(30,1,15) vs valeurs réelles du nombre de conteneurs

On constate que la courbe de prévision suit globalement la tendance des observations, mais les écarts restent très significatifs dû aux sauts brusques du flux.

Le modèle Random Forest Face aux limites des modèles ARIMA, nous avons développé un modèle *Random Forest* en procédant à un enrichissement des données par création de variables temporelles dérivées :
- **Lags** : valeurs de la série décalées de 1 à k pas de temps ;
- **Rolling mean** : moyennes mobiles sur des fenêtres de taille w ;
- **Saisonnalité** : composantes cycliques via fonctions trigonométriques $\cos(2\pi t/s)$ et $\sin(2\pi t/s)$ pour capturer les effets hebdomadaires et mensuels ;
- **Indicateurs calendaires** : jour de semaine, mois, trimestre, etc.

Le modèle a obtenu une **MAE** de 0,2, nettement plus faible qu'avec ARIMA$(30, 1, 15)$, tout en suivant globalement la tendance observée :

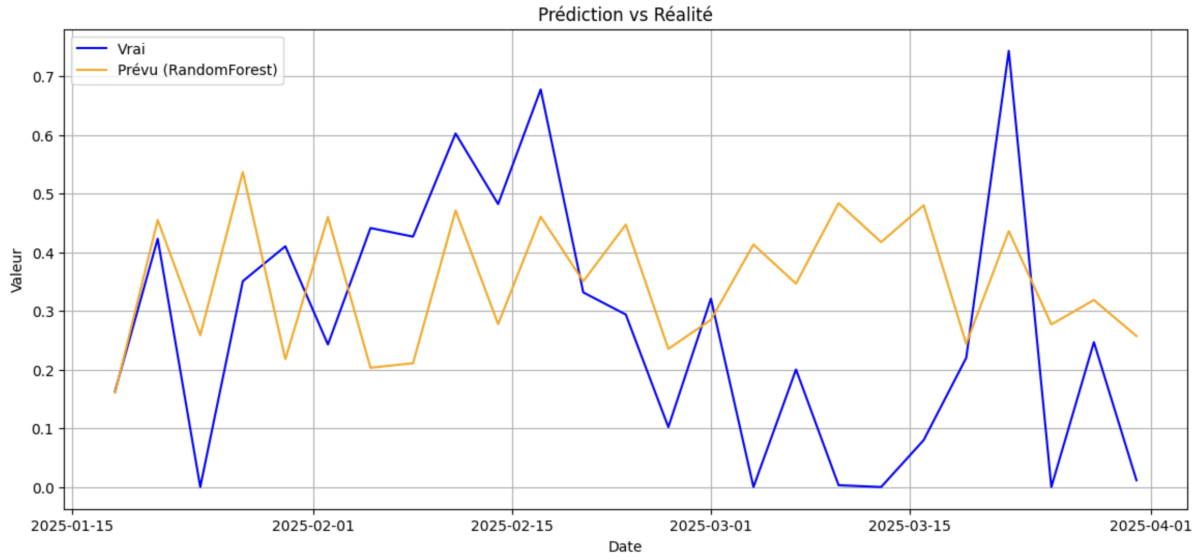


FIGURE 5.12 – Prévisions Random Forest vs valeurs réelles du nombre de conteneurs

Conclusion Malgré une analyse approfondie des séries et une modélisation adaptée, les prévisions demeurent imprécises. Avec un MAE de 0,36, ce modèle n'est pas utilisable en l'état. Le principal problème est le manque de données : une seule année d'observations ne permet pas de saisir les variations soudaines, ce qui génère des prévisions contre-tendance.

5.5 Cas d'usage 3 : Prévision du nombre de dossiers

Objectif

Prédire le nombre de dossiers qui seront soumis par jour.

Gain attendu

— Optimisation de la charge de travail ;
— Meilleure gestion des tâches.

Exploration des données (EDA)

Structure du jeu de données

— **Nombre d'observations** : 455
— **Fréquence** : journalier
— **Nombre de variables** : 2
— **Variable cible** : `NB_DOSSIER`

Prétraitement des données

— Remplissage des valeurs manquantes, généralement le dimanche car aucun dossier n'est enregistré, donc on met 0 sur les NaN.

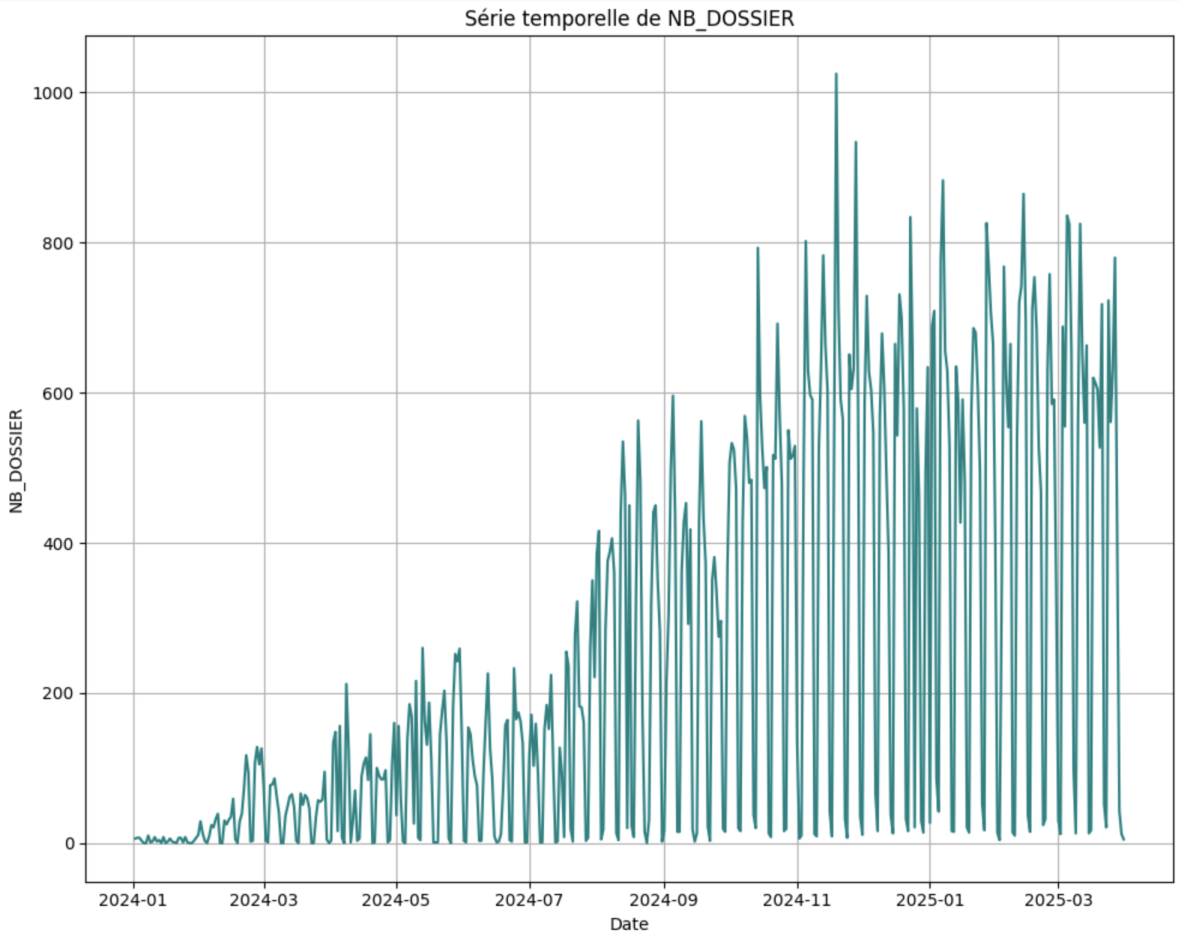


FIGURE 5.13 – Nombre de dossiers par jour en fonction du temps

Décomposition de la série

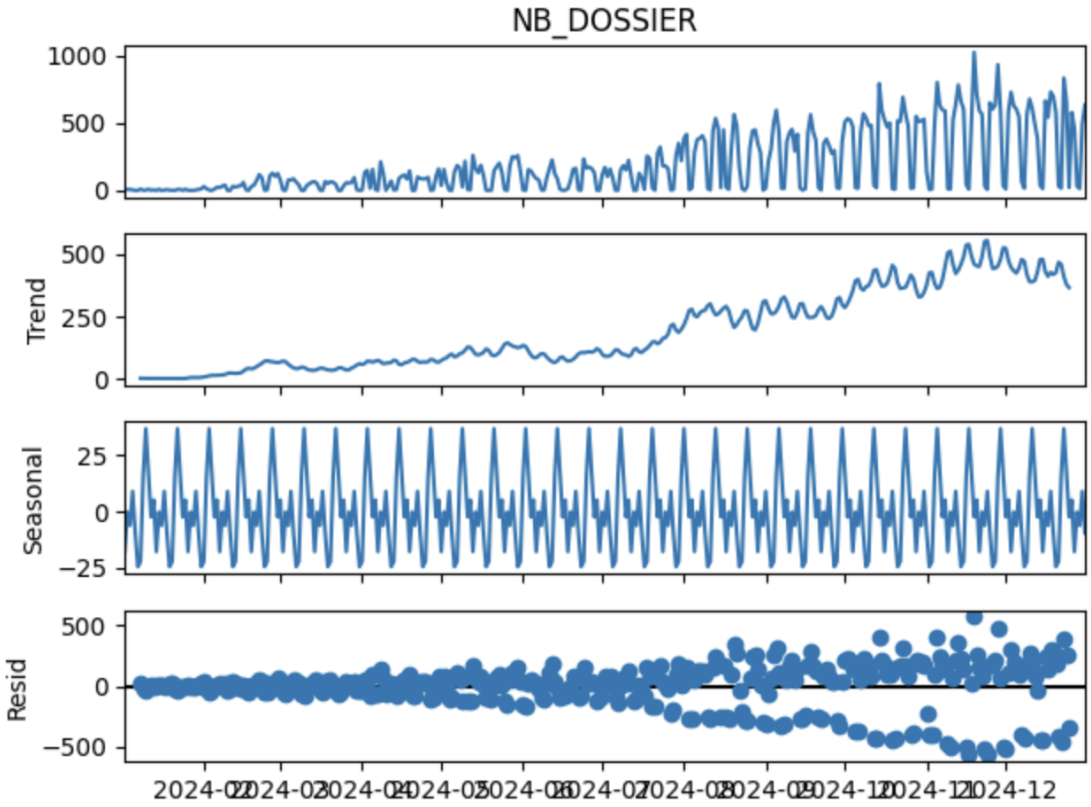


FIGURE 5.14 – Décomposition de la série temporelle

Diagnostic de la série L'observation de la série montre qu'elle ne présente pas de tendance marquée à long terme, tandis que la saisonnalité est immédiatement visible. Cela suggère d'emblée l'utilisation d'un modèle prenant en compte la composante saisonnière, tel que SARIMA. De plus, l'analyse des résidus indique qu'ils suivent approximativement une distribution normale, validant en partie les hypothèses de bruit blanc du modèle.

Analyse de la dépendance temporelle

- On trace l'**ACF** (fonction d'autocorrélation) pour observer la corrélation entre y_t et y_{t-k} ;
- puis la **PACF** (fonction d'autocorrélation partielle) pour estimer la dépendance directe avec les retards.

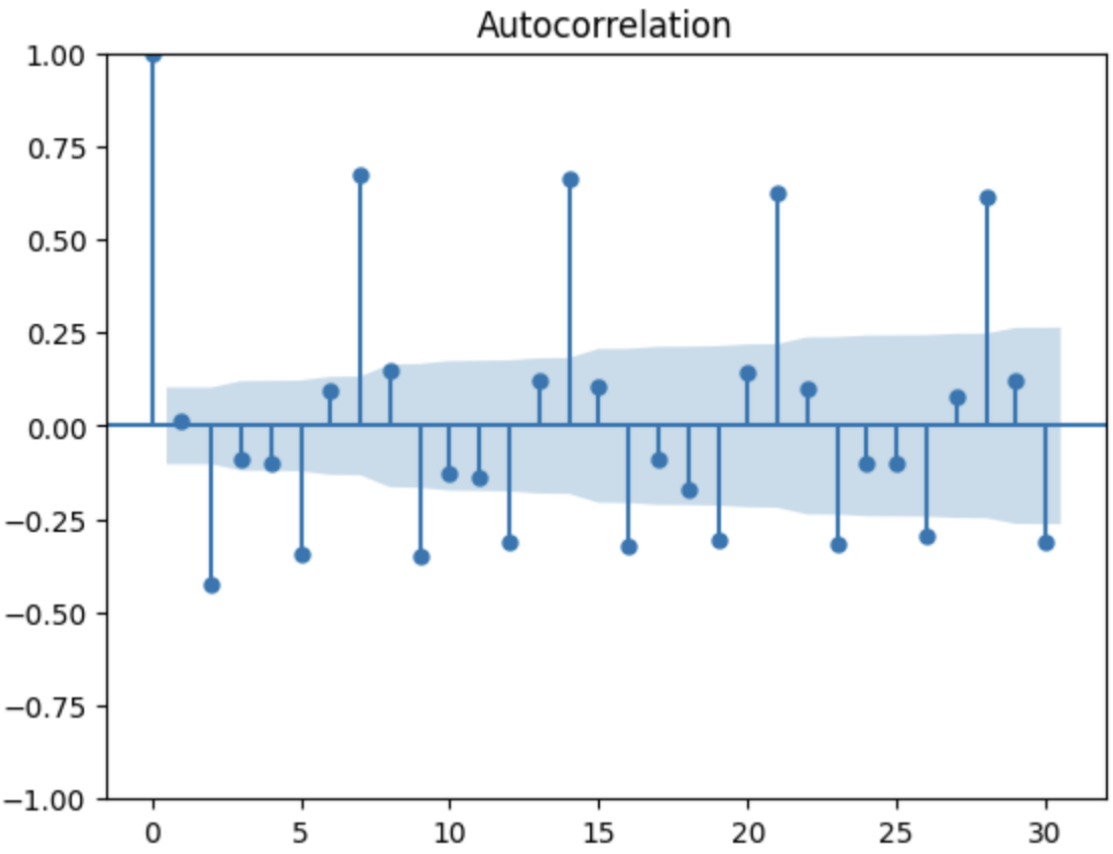


FIGURE 5.15 – Courbe d'autocorrélation des lags

Le fait que les pics restent importants et réguliers tout au long de la courbe confirme la saisonnalité de la série. Les mêmes valeurs se répètent tous les s unités de temps, et dans notre cas, la saisonnalité est hebdomadaire.

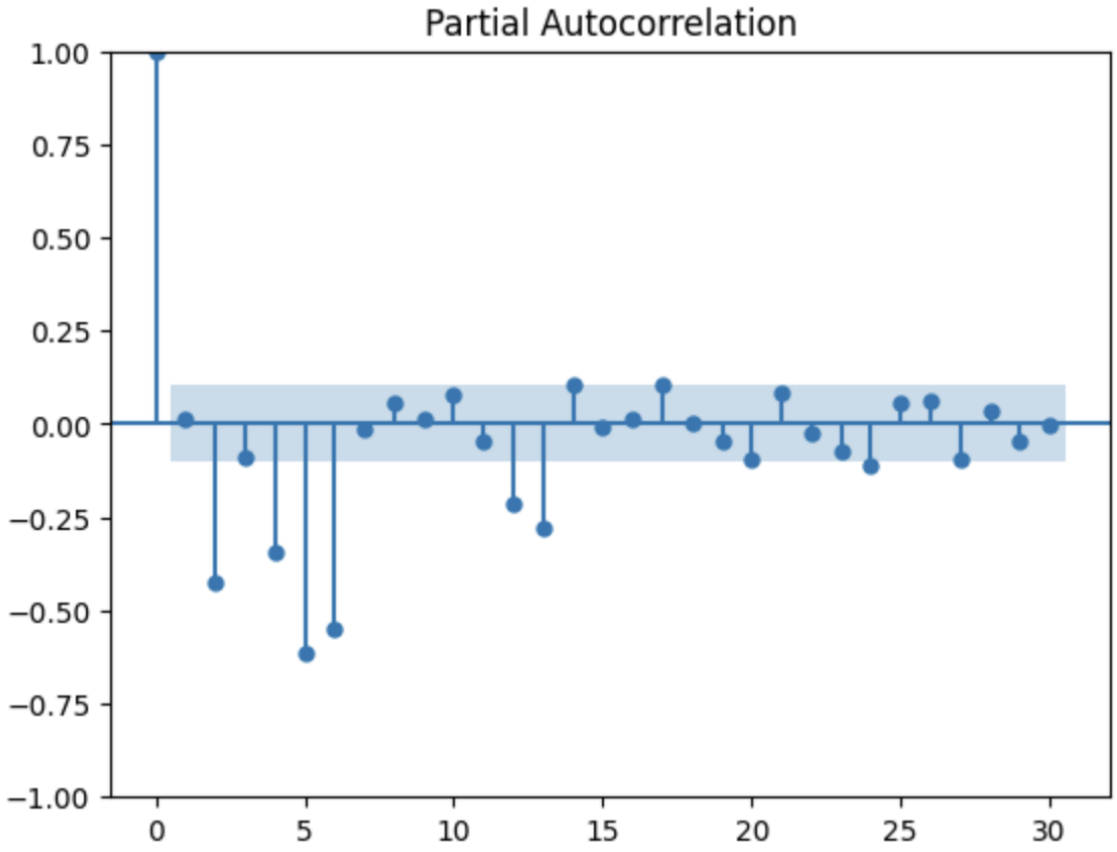


FIGURE 5.16 – Courbe d'autocorrélation partielle des lags

La courbe PACF quant à elle n'a pas de structure régulière interprétable. Mais on est sûr de n'avoir ni affaire à un simple modèle MA ou AR, donc un modèle SARIMA serait le plus adéquat.

Modélisation

Le modèle SARIMA(p,d,q)(P,D,Q)$_s$ Le modèle SARIMA étend ARIMA en intégrant explicitement une composante saisonnière de période s. Il se formule ainsi :

$$\Phi(L^s)\,\phi(L)\,(1-L)^d\,(1-L^s)^D\,y_t = \Theta(L^s)\,\theta(L)\,\varepsilon_t,$$

où

$$\phi(L) = 1 - \phi_1 L - \cdots - \phi_p L^p, \quad \Phi(L^s) = 1 - \Phi_1 L^s - \cdots - \Phi_P L^{Ps},$$
$$\theta(L) = 1 + \theta_1 L + \cdots + \theta_q L^q, \quad \Theta(L^s) = 1 + \Theta_1 L^s + \cdots + \Theta_Q L^{Qs},$$

et $(1-L)^d$, $(1-L^s)^D$ sont les opérateurs de différenciation non saisonnière et saisonnière.

Sélection des ordres (p,d,q) et (P,D,Q)
- Utiliser l'ACF et la PACF pour estimer :
  - p et q à partir des comportements aux petits lags,
  - P et Q aux lags multiples de la saison s,
  - d et D via les tests de stationnarité (ADF/KPSS) avant et après différenciation.

— Comparer plusieurs modèles SARIMA$(p,d,q)(P,D,Q)_s$ à l'aide de critères d'information, puis retenir la combinaison qui minimise l'**AIC** :

$$\mathrm{AIC} = -2\ell(\widehat{\theta}) + 2K,$$

où $\ell(\widehat{\theta})$ est la log-vraisemblance évaluée au maximum et K le nombre de paramètres.

Prévisions Le modèle prédit avec une MAE de 0,67 et la courbe de prévision est la suivante :

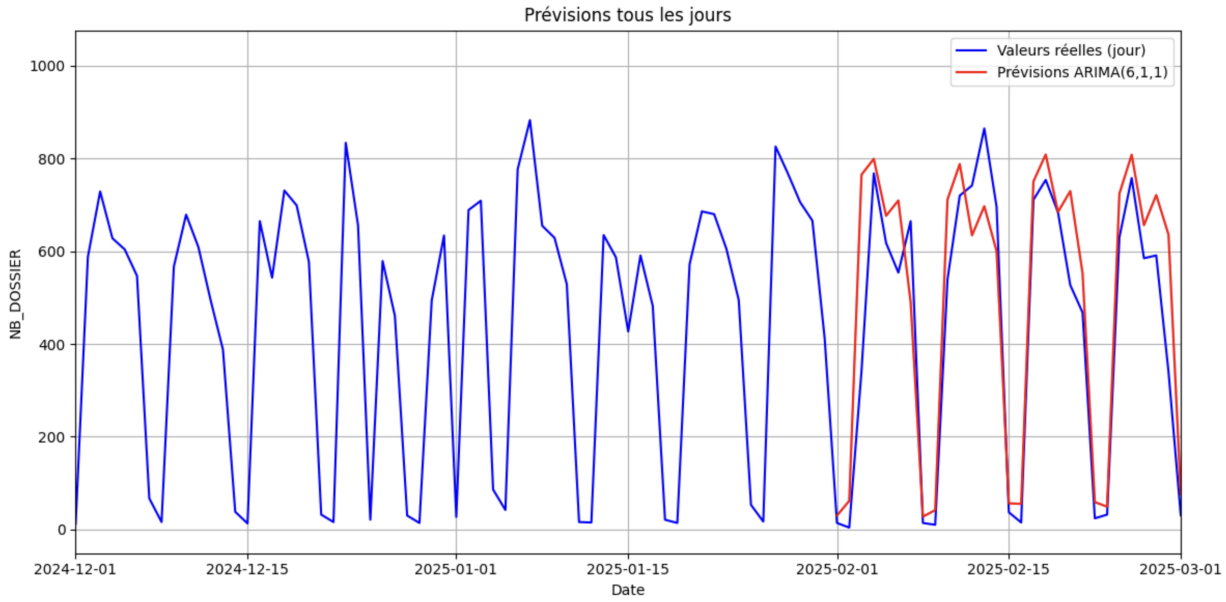


FIGURE 5.17 – Prévisions SARIMA(6,1,1)(3,1,4,7) vs valeurs réelles du nombre de dossiers

Diagnostic des résidus

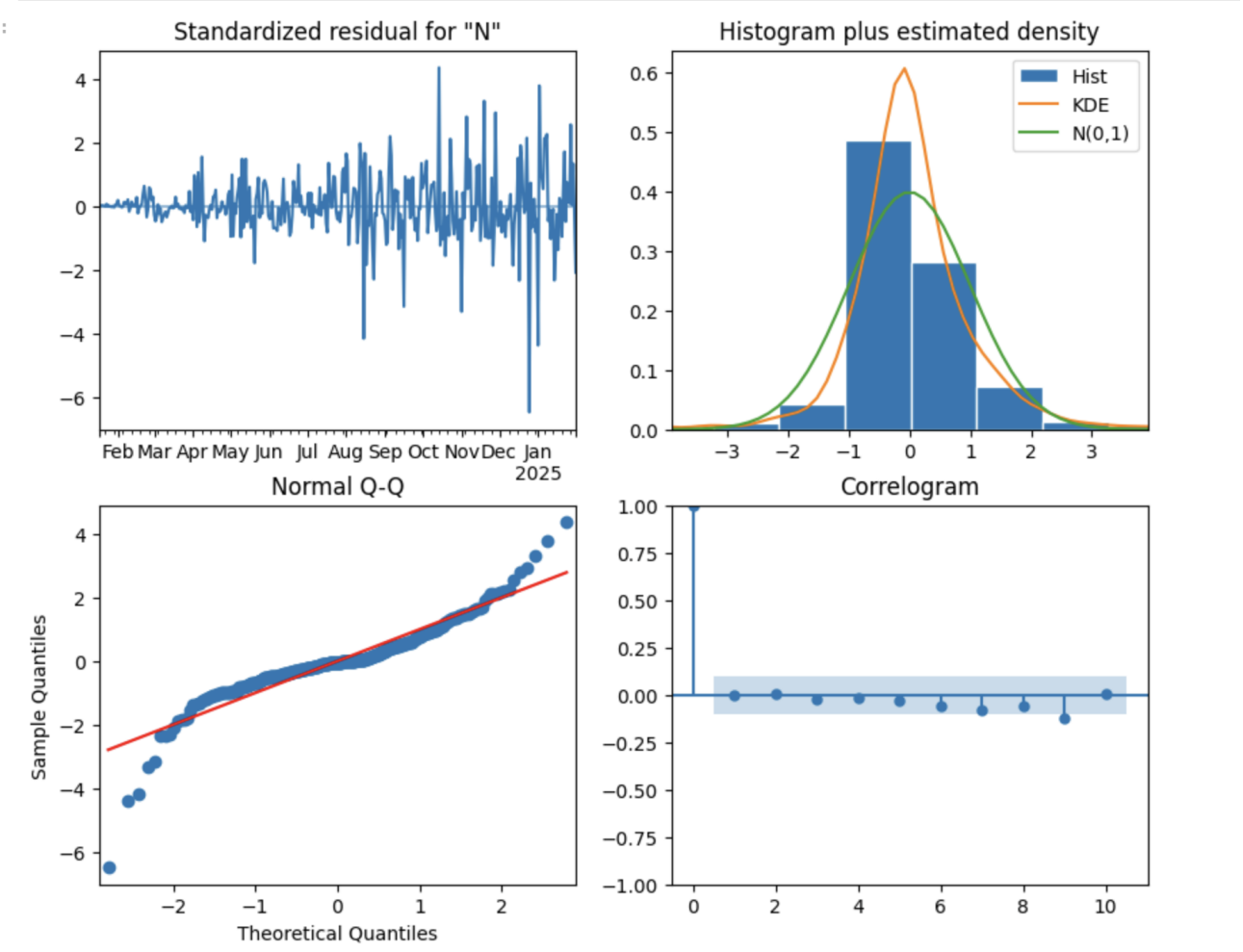


FIGURE 5.18 – Résidus du modèle SARIMA

Les résidus suivent une distribution proche de la normale, donc ils peuvent être considérés comme du bruit blanc.

Intégration dans Oracle APEX

Après la phase de modélisation, le modèle SARIMA a été déployé sous forme d'API `FastAPI`. L'endpoint `/predict` reçoit en entrée le nombre de pas de prévision et renvoie un JSON :

```
POST /predict
Body : {"num_steps": 5}
Réponse : {"prediction":[29.85,61.67,765.30,799.28,676.45]}
```

Côté APEX, l'appel se fait via la fonction `APEX_WEB_SERVICE.make_rest_request` :

```
l_resp := APEX_WEB_SERVICE.make_rest_request(
  p_url => 'http://<serveur>/predict',
  p_http_method => 'POST',
  p_body => '{"num_steps": ' || :P1_INPUT || '}');
```

La réponse JSON est ensuite parsée avec `APEX_JSON` pour alimenter les graphiques et rapports interactifs de l'application. Cette intégration permet aux utilisateurs de lancer des prévisions en temps réel directement depuis l'interface APEX.

Conclusion

En somme, après une analyse de la série et une modélisation adaptée à cette dernière, on a obtenu des prédictions d'une précision proche de 70 %. Le modèle est plus performant sur un regroupement par trois jours du nombre de dossiers, mais ce cas de figure reste moins intéressant.

5.5.1 Discussion sur les contributions

Les solutions implémentées adressent directement les problématiques métiers décrites dans la section 1.3 :

- **Cas d'usage 1** : Le modèle Random Forest permet une détection précoce des dossiers à risque, réduisant les délais de traitement et optimisant l'allocation des ressources humaines. Les seuils de probabilité (80% et 50%) facilitent la priorisation des vérifications, fluidifiant la chaîne de traitement douanier.
- **Cas d'usage 2** : Le modèle Random Forest (MAE de 0,2) offre une base pour anticiper les volumes de conteneurs, réduisant les congestions aux quais et améliorant la planification des ressources. Cependant, le manque de données (une année d'observations) limite la précision.
- **Cas d'usage 3** : Le modèle SARIMA (MAE de 0,67) permet une estimation raisonnable du flux de dossiers, facilitant la gestion des charges de travail. Une précision accrue sur des agrégations tri-journalières suggère une adaptation possible pour des horizons de planification plus larges.

Ces modèles transforment le patrimoine de données de Gaïndé 2000 en outils d'aide à la décision, renforçant l'efficacité opérationnelle du Port Autonome de Dakar. Leur intégration dans un tableau de bord interactif et l'acquisition de données supplémentaires sont nécessaires pour un déploiement opérationnel.

Chapitre 6

Conclusion et perspective

6.1 Conclusion

Ce stage a permis de valoriser les données historiques de Gaïndé 2000 pour répondre à trois problématiques métiers critiques : la classification des dossiers douaniers, la prévision des volumes de marchandises, et l'estimation du flux de dossiers. Les modèles développés (**Random Forest** pour la classification, **ARIMA** et **Random Forest** pour les volumes de marchandises, **SARIMA** pour les dossiers) ont atteint des performances prometteuses, avec un F1-score de 0,96 pour la classification et des MAE de 0,2 à 0,67 pour les prévisions. Ces résultats démontrent le potentiel des techniques d'intelligence artificielle pour optimiser les processus portuaires, en réduisant les délais, les congestions et les coûts opérationnels.

6.2 Leçon apprise

L'analyse approfondie des données et la modélisation ont révélé plusieurs enseignements :

— L'importance d'un prétraitement rigoureux (gestion des valeurs manquantes, standardisation des formats) pour garantir la qualité des modèles.
— La robustesse de Random Forest pour la classification, même avec des données bruitées, grâce à sa capacité à capturer des interactions non linéaires.
— Les limites des modèles de séries temporelles (ARIMA, SARIMA) face à un faible volume de données, soulignant le besoin d'historiques plus longs.
— L'efficacité des variables dérivées (lags, moyennes mobiles) pour améliorer les performances des modèles Random Forest sur des séries temporelles.

6.3 Perspective

Pour améliorer les solutions développées et les rendre opérationnelles, les pistes suivantes sont envisagées :

— **Collecte de données supplémentaires** : Un historique plus long (plusieurs années) permettrait de mieux capturer les variations saisonnières et d'améliorer la précision des prévisions.
— **Intégration dans un tableau de bord** : Déployer les modèles dans une interface utilisateur sur Oracle APEX pour une utilisation en temps réel par les équipes portuaires.
— **Exploration de modèles avancés** : Tester des approches comme LSTM ou Prophet pour les séries temporelles, afin de capturer des dynamiques non linéaires complexes.

— **Automatisation du pipeline** : Mettre en place un système automatisé pour le prétraitement, l'entraînement et la mise à jour des modèles avec de nouvelles données.

Ces étapes permettront de renforcer l'impact des solutions sur l'efficacité opérationnelle et de soutenir la transformation numérique du Port Autonome de Dakar.

Annexe A

DD&RS - Développement Durable et Responsabilité Sociétale

A.1 Impacts environnementaux

A.1.1 Actions liées au stage et aux stratégies de l'entreprise

Pendant le stage, des efforts ont été faits pour optimiser le prétraitement des données et l'entraînement des modèles afin de réduire les besoins computationnels, en cohérence avec les objectifs de durabilité de l'entreprise.

Annexe B

Structure de l'entreprise

La structure organisationnelle de Bangath Systems est une structure à taille humain composé de :

— **Département Data Science et Intelligence Artificielle** : Responsable de l'analyse des données, du développement de modèles et de leur intégration dans des plateformes comme Oracle APEX.

— **Département Infrastructure Informatique** : Gère l'environnement Oracle Cloud et les systèmes de bases de données et la conception d'application.

— **Bureau de Gestion de Projets** : Coordonne les partenariats, notamment avec Gaïndé 2000 et ses partenaires dont le Port Autonome de Dakar et la Douane sénégalaise.

Annexe C

Détails techniques

C.1 Extraits de code supplémentaires

C.1.1 Cas d'usage 1 : Estimation de la probabilité de succès d'un dossier

Ce cas d'usage vise à assigner une probabilité de succès à chaque déclaration pour un traitement sans renvoi ni blocage, optimisant ainsi l'allocation des ressources et la réactivité opérationnelle.

Prétraitement des données

La fonction `data_cleaning` effectue un nettoyage préalable des données, incluant la suppression des colonnes à faible variance, la conversion des types de données, et la gestion des colonnes à haute cardinalité.

```
def data_cleaning(df):
    # Supprimer les colonnes num riques avec une seule valeur unique
    nunique = df.select_dtypes(include='number').nunique()
    cols_to_drop = nunique[nunique == 1].index.tolist()
    df.drop(columns=cols_to_drop, inplace=True)

    # Convertir les colonnes num riques avec deux valeurs uniques en bool en
    boolean_cols = nunique[nunique == 2].index.tolist()
    df[boolean_cols] = df[boolean_cols].astype('bool')

    # Correction de types sp cifiques
    if 'PAYED_FRAIS_BAD' in df.columns:
        df['PAYED_FRAIS_BAD'] = df['PAYED_FRAIS_BAD'].astype('int', errors='ignore')

    # Supprimer les colonnes objets avec trop de modalit s
    object_cols = df.select_dtypes(include='object')
    high_card_cols = [col for col in object_cols if df[col].nunique() >= 3674]
    df.drop(columns=high_card_cols, inplace=True)

    # Supprimer une colonne sp cifique si elle existe
    if 'DATE_VALIDITE_CONNAISSEMENT' in df.columns:
        df.drop('DATE_VALIDITE_CONNAISSEMENT', axis=1, inplace=True)

    # Conversion de la colonne ART en entier
    if 'ART' in df.columns:
        df['ART'] = df['ART'].astype('int', errors='ignore')

    # Supprimer les colonnes avec moins de deux modalit s
```

```
    for col in df.columns:
        if df[col].nunique() < 2:
            df.drop(col, axis=1, inplace=True)

    # Convertir les entiers en float
    for col in df.select_dtypes('int').columns:
        df[col] = df[col].astype('float')

    # Reconvertir les bool ens en int
    for col in df.select_dtypes('bool').columns:
        df[col] = df[col].astype('int')

    df = df.drop(columns=[col for col in df.columns if df[col].value_counts().max() <
    2])

    return df
```

Listing C.1 – Fonction de nettoyage des données

Use Case 1 : Random Forest (ADS / SklearnModel)

```
import ads
from ads.model.framework.sklearn_model import SklearnModel
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# 1. D finition des variables explicatives X et de la cible y
X = df_final[[
    "SENDED_DEMANDE_TO_CONSIGNATAIRE",
    "PAYED_FRAIS_BAD",
    "DELIVRED_DOCUMENT_BON_CONSIGNATAIRE",
    "DOSSIER_IS_TRANSFERT_TO_OTHER_TRANSITAIRE",
    "SEND_DEMANDE_TO_TRANSPORTEUR",
    "DEMANDE_IS_SEND_TO_DAKAR_TERMINAL_FORMBAD",
    "QUITUS_FRAIS_GUICHET_UNIQUE",
    "POIDS",
    "NOMBRE_COLIS"
]]
y = df_final['DOSSIER_IS_REGULE']  # 0  : bloqu , 1  : r gularis

# 2. S paration train / test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# 3. Entra nement du mod le Random Forest
rf_model = RandomForestClassifier(
    n_estimators=100,
    class_weight="balanced",  # pour d s quilibre des classes
    random_state=42
)
rf_model.fit(X_train, y_train)

# 4. Pr dictions
y_pred = rf_model.predict(X_test)
y_proba = rf_model.predict_proba(X_test)
```

```
37
38 # 5.    valuation
39 print("=== Matrice de confusion ===")
40 print(confusion_matrix(y_test, y_pred))
41 print("\n=== Rapport de classification ===")
42 print(classification_report(y_test, y_pred, digits=3))
```

Listing C.2 – Use Case 1 : Random Forest via ADS / Scikit-learn

Use Case 2 : Diagnostic ARIMA et optimisation VARMAX / SARIMA

```
1 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
2 from statsmodels.tsa.seasonal import seasonal_decompose
3 from statsmodels.tsa.stattools import adfuller
4 import matplotlib.pyplot as plt
5
6 def my_arima(cible, d, ts):
7     if d:
8         ts = df_train[cible].diff().dropna()
9
10    # D composition
11    decomp = seasonal_decompose(ts, model='additive', period=12)
12    decomp.plot()
13    plt.show()
14
15    # Test de stationnarit  ADF
16    result = adfuller(ts)
17    ADF, p_value = result[0], result[1]
18    print('ADF statistic:', ADF)
19    print('p-value:', p_value)
20
21    # Trac  ACF / PACF si stationnaire
22    if p_value < 0.05:
23        plot_acf(ts, lags=30)
24        plot_pacf(ts, lags=30)
25        plt.show()
26    else:
27        my_arima(cible, d+1, ts)
28        print("Degr  de diff renciation :", d+1)
```

Listing C.3 – Use Case 2 : Diagnostic ARIMA

```
1 import pandas as pd
2 import numpy as np
3 from typing import Union
4 from statsmodels.tsa.statespace.varmax import VARMAX
5 from itertools import product
6 from tqdm.notebook import tqdm
7
8 def optimize_VAR(endog: Union[pd.DataFrame, list],
9                  p_range=range(0, 6), q_range=range(0, 6)) -> pd.DataFrame:
10    """
11    Recherche des ordres (p,q) optimaux pour un mod le VARMAX
12    en minimisant l'AIC.
13    """
14    results = []
15    for p, q in tqdm(product(p_range, q_range),
```

```
                        total=len(p_range)*len(q_range),
                        desc='VARMAX grid search'):
        try:
            model = VARMAX(endog, order=(p, q))
            fit = model.fit(disp=False)
            results.append({'p': p, 'q': q, 'AIC': fit.aic})
        except Exception:
            continue

    return pd.DataFrame(results).sort_values(by='AIC').reset_index(drop=True)
```

Listing C.4 – Use Case 2 : Optimisation VARMAX

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

p, d, q = 30, 1, 15
model = SARIMAX(ts0, order=(p, d, q), simple_differencing=False)
model_fit = model.fit()
```

Listing C.5 – Use Case 2 : Exécution SARIMAX fixe

Use Case 3 : Feature Engineering pour séries temporelles

```
import pandas as pd
import numpy as np

# ts0 : s rie index e en datetime
ts0 = df_final[cible]

# Initialisation du DataFrame
df_feat = pd.DataFrame(index=ts0.index)
df_feat[cible] = ts0

# LAG features
lags = [7]
for lag in lags:
    df_feat[f'lag{lag}'] = ts0.shift(lag)

# Rolling statistics
windows = [3, 6, 7, 30]
for w in windows:
    shifted = ts0.shift(1)
    df_feat[f'roll_mean_{w}'] = shifted.rolling(w).mean()
    df_feat[f'roll_std_{w}']  = shifted.rolling(w).std()
    df_feat[f'roll_min_{w}']  = shifted.rolling(w).min()
    df_feat[f'roll_max_{w}']  = shifted.rolling(w).max()

# Composantes calendrier
df_feat['month']        = ts0.index.month
df_feat['day']          = ts0.index.day
df_feat['week']         = ts0.index.isocalendar().week
df_feat['weekday']      = ts0.index.weekday
df_feat['is_weekend']   = df_feat['weekday'].isin([5,6]).astype(int)
df_feat['is_start_month'] = (ts0.index.day == 1).astype(int)
end_of_month = ts0.index.to_series().groupby(ts0.index.to_period('M')).transform('max')
df_feat['is_end_month']   = (ts0.index == end_of_month).astype(int)

```

```
# Encodage cyclique
df_feat['month_sin']   = np.sin(2*np.pi*df_feat['month']/12)
df_feat['month_cos']   = np.cos(2*np.pi*df_feat['month']/12)
df_feat['weekday_sin']= np.sin(2*np.pi*df_feat['weekday']/7)
df_feat['weekday_cos']= np.cos(2*np.pi*df_feat['weekday']/7)

# Finalisation
df_feat = df_feat.dropna()
```

Listing C.6 – Use Case 3 : Génération de features temporelles

Bibliographie

[1] Scikit-learn : Apprentissage automatique en Python.
https://scikit-learn.org

[2] Statsmodels : Modélisation statistique en Python.
https://www.statsmodels.org

[3] Documentation Oracle Cloud Data Science.
https://docs.oracle.com/en/cloud/paas/data-science

[4] Marco Peixeiro. Time Series Forecasting in Python.
https://towardsdatascience.com/time-series-forecasting-with-machine-learning-in-python-3b4